

Chapter 1

Computer Abstractions and Technology



The Computer Revolution

- Progress in computer technology
 - Underpinned by domain-specific accelerators
- Makes novel applications feasible
 - Computers in automobiles
 - Cell phones
 - Human genome project
 - World Wide Web
 - Search Engines
- Computers are pervasive

Classes of Computers

■ Personal computers

- General purpose, variety of software
- Subject to cost/performance tradeoff

■ Server computers

- Network based
- High capacity, performance, reliability
- Range from small servers to building sized

Classes of Computers

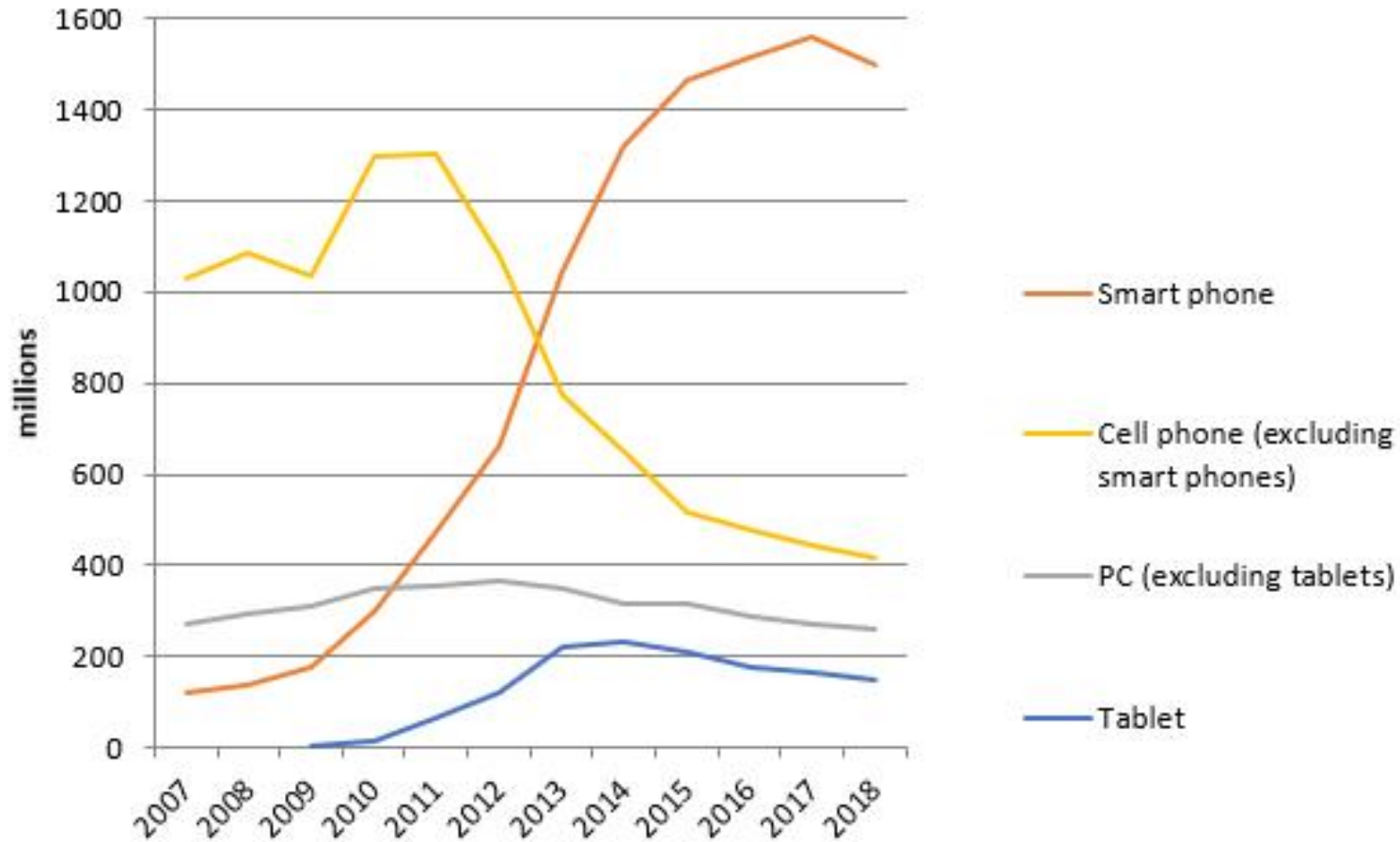
■ Supercomputers

- Type of server
- High-end scientific and engineering calculations
- Highest capability but represent a small fraction of the overall computer market

■ Embedded computers

- Hidden as components of systems
- Stringent power/performance/cost constraints

The PostPC Era

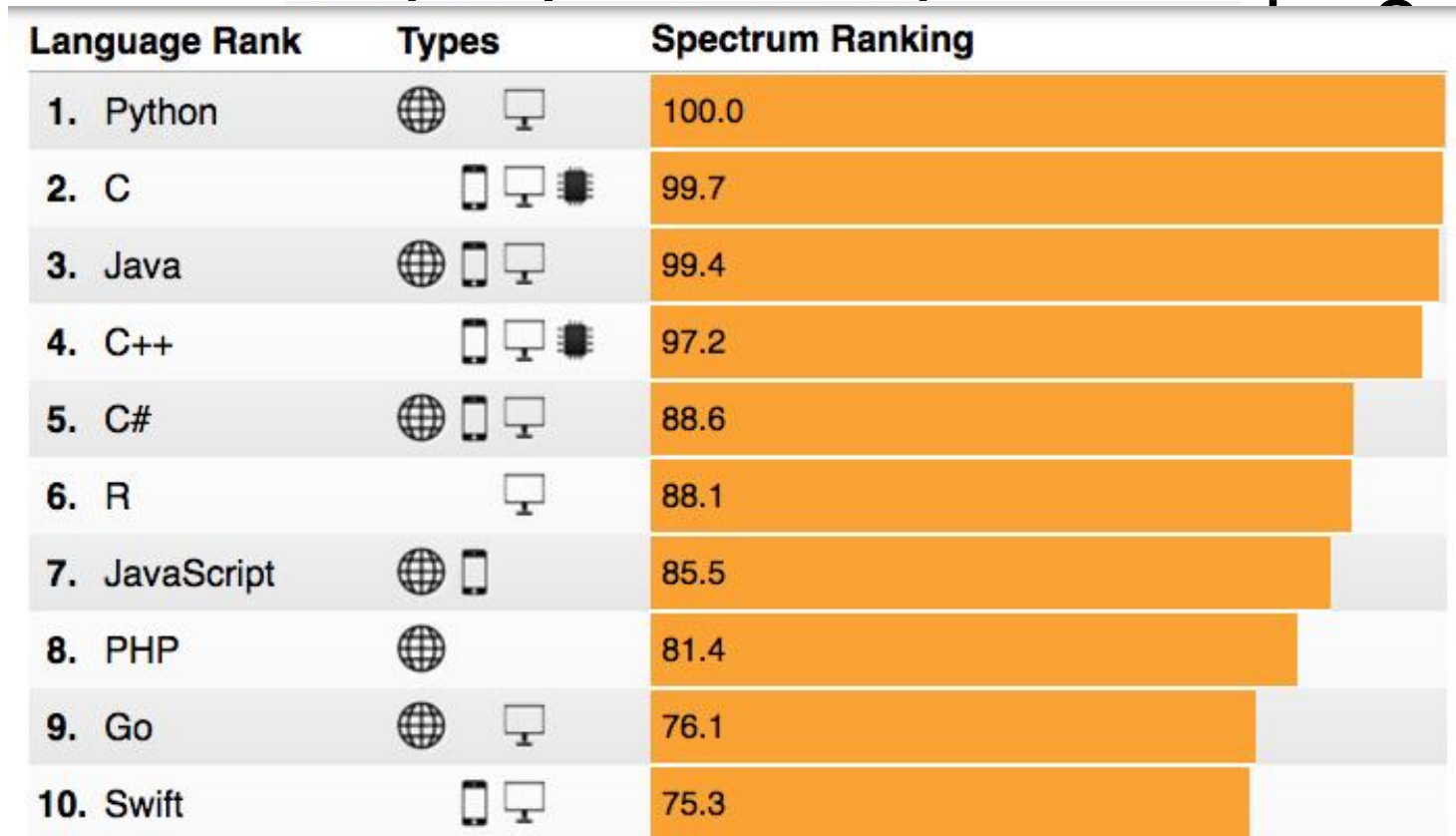


The PostPC Era

- Personal Mobile Device (PMD)
 - Battery operated
 - Connects to the Internet
 - Hundreds of dollars
 - Smart phones, tablets, electronic glasses,bracelet
- Cloud computing
 - Warehouse Scale Computers (WSC)
 - Software as a Service (SaaS)
 - Portion of software run on a PMD and a portion run in the Cloud
 - Amazon and Google

Most Popular Programming Languages 2016-7

- What do you think is the most “popular”



Why You Need to Learn C!

This course is NOT really about C Programming

- It is about the *hardware-software interface*
 - What does the programmer need to know to achieve the highest possible performance
- C is close to the underlying hardware, unlike languages like Python and Java!
 - Allows us to talk about key hardware features in higher level terms
 - Allows programmer to explicitly harness underlying hardware parallelism for higher *performance* and *power efficiency*

Software

Hardware

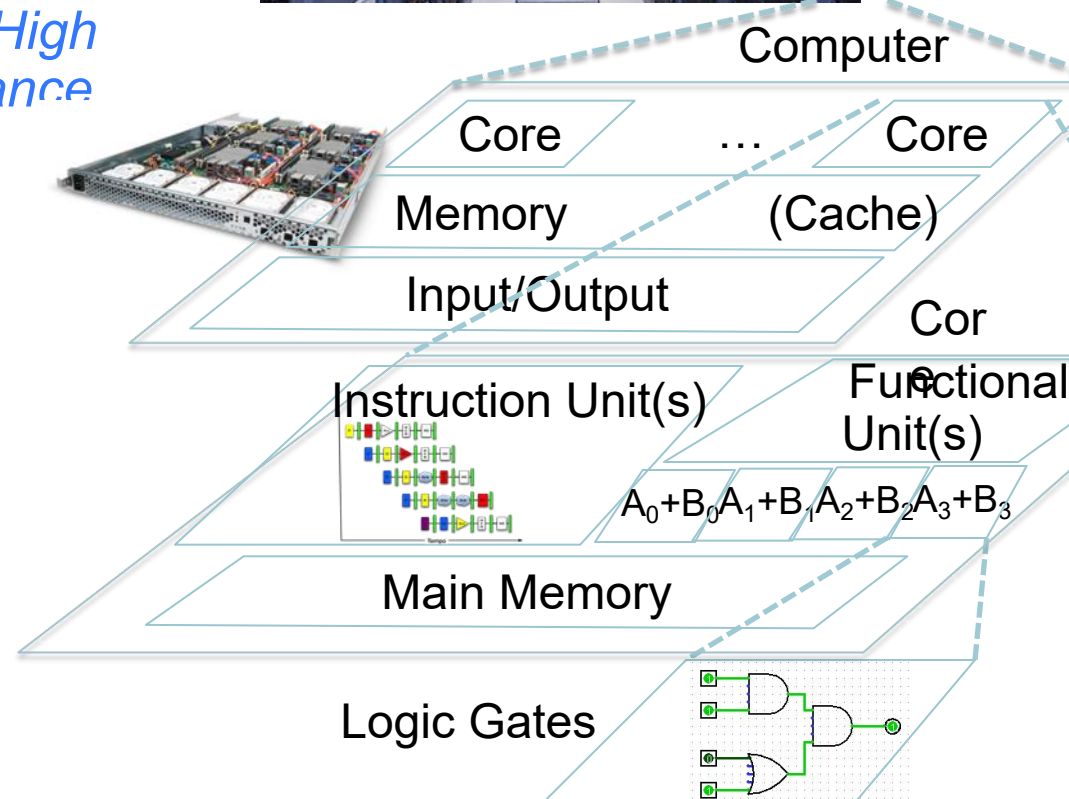
- **Parallel Requests**
Assigned to computer
e.g., Search “cats”
- **Parallel Threads**
Assigned to core
e.g., Lookup, Ads
- **Parallel Instructions**
>1 instruction @ one time
e.g., 5 pipelined instructions
- **Parallel Data**
>1 data item @ one time
e.g., Add of 4 pairs of words
- **Hardware descriptions**
All gates functioning in parallel
at same time

*Harness
Parallelism &
Achieve High
Performance*

Warehouse-
Scale
Computer



Smart
Phone



What You Will Learn In This Course

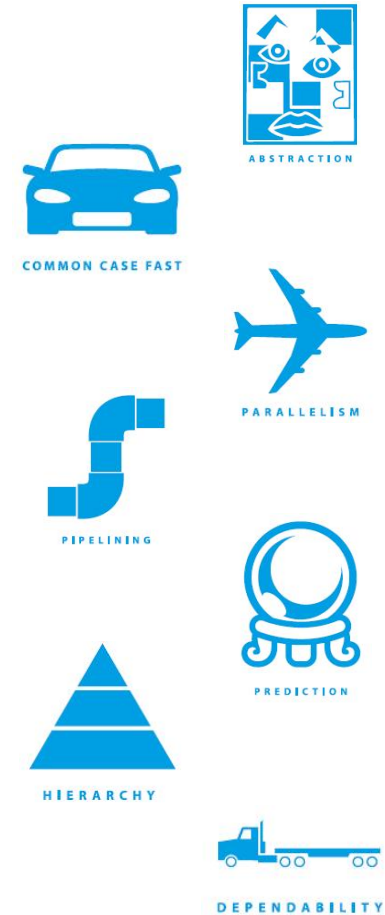
- How programs are translated into the machine language
 - And how the hardware executes them
- The hardware/software interface
- What determines program performance
 - And how it can be improved
- How hardware designers improve performance
- What is parallel processing

Understanding Performance

- **Algorithm**
 - Determines number of operations executed
- **Programming language, compiler, architecture**
 - Determine number of machine instructions executed per operation
- **Processor and memory system**
 - Determine how fast instructions are executed
- **I/O system (including OS)**
 - Determines how fast I/O operations are executed

Seven Great Ideas

- Use **abstraction** to simplify design
- **Hierarchy** of memories
- Performance via **pipelining**
- Performance via **parallelism**
- **Dependability** via redundancy
- Make the **common case fast**
- Performance via **prediction**



Great Idea #1: Abstraction

(Levels of Representation/Interpretation)

High Level Language
Program (e.g., C)

Compiler

Assembly Language
Program (e.g., MIPS)

Assembler

Machine Language
Program (MIPS)

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
lw    $t0, 0($2)
lw    $t1, 4($2)
sw    $t1, 0($2)
sw    $t0, 4($2)
```

Anything can be represented
as a *number*,
i.e., data or instructions

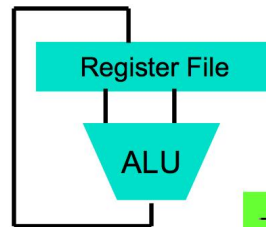
```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 1000 0000 1001
0101 1000 0000 1001 1100 0110 1010 1111
```

Machine Interpretation

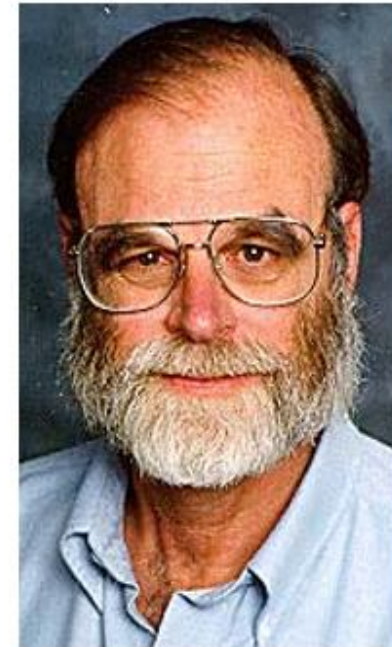
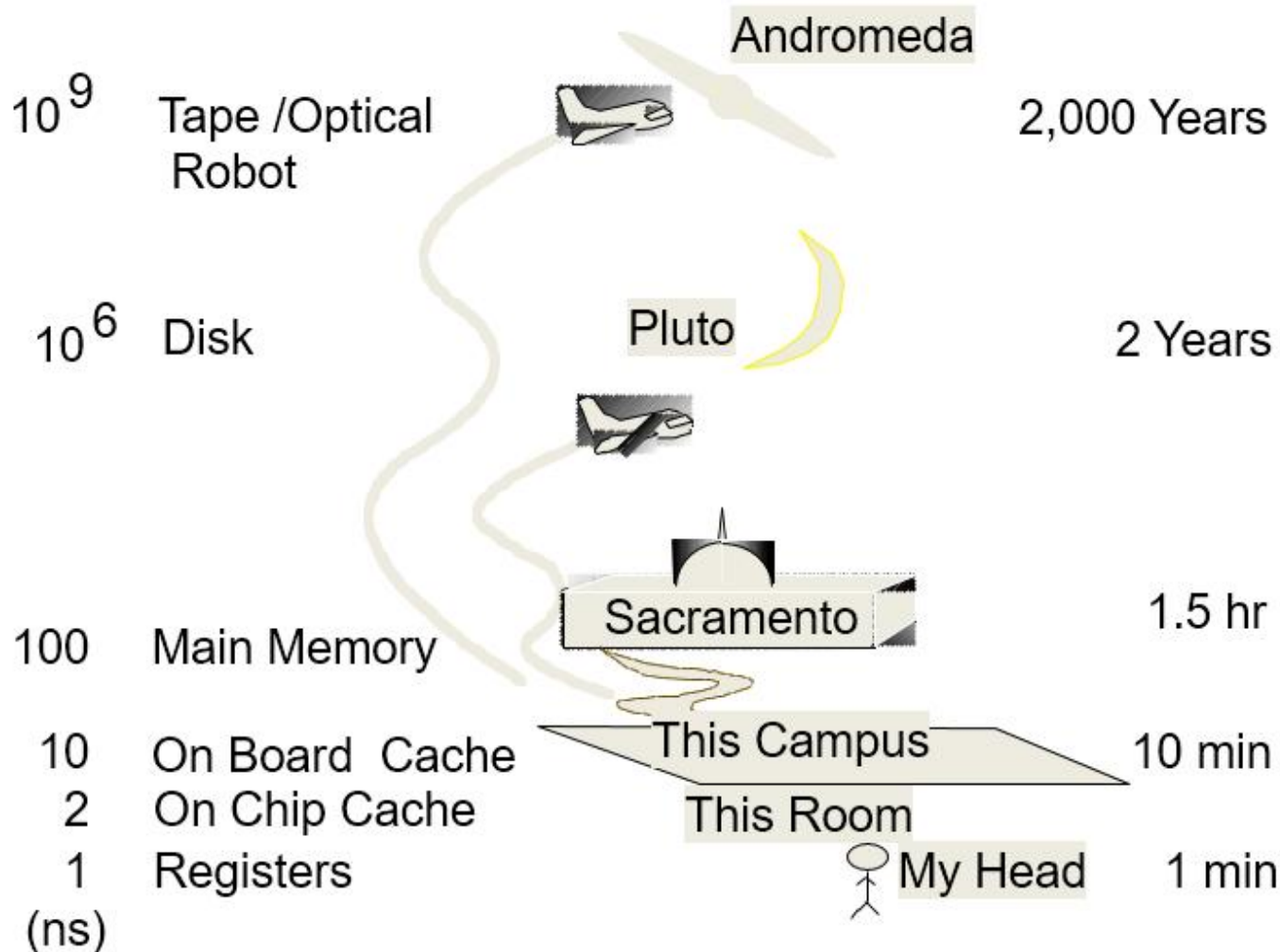
Hardware Architecture Description
(e.g., block diagrams)

Architecture Implementation

Logic Circuit Description
(Circuit Schematic Diagrams)

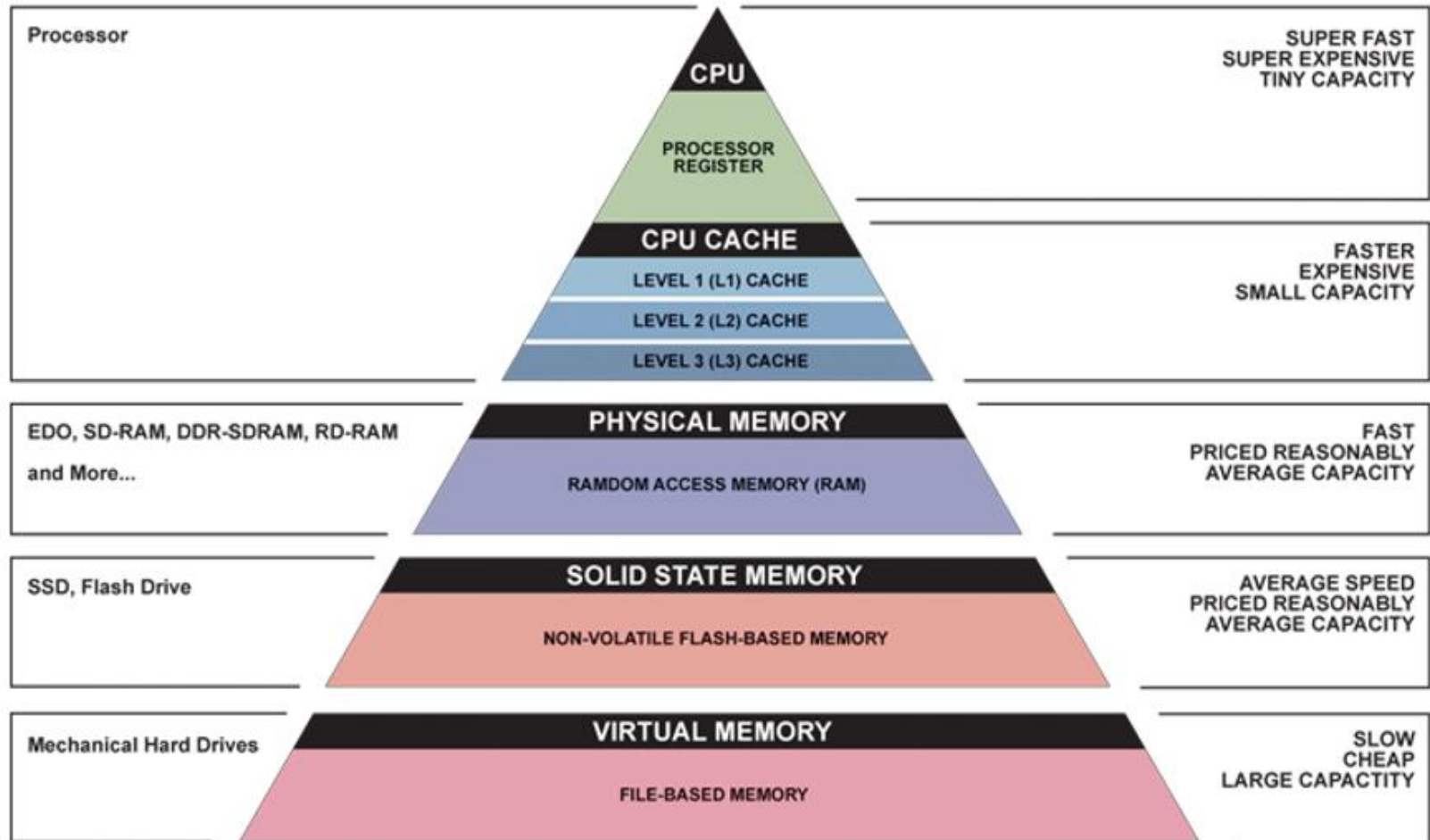


Jim Gray's Storage Latency Analogy: How Far Away is the Data?



Jim Gray
Turing Award
B.S. Cal 1966
Ph.D. Cal 1969!

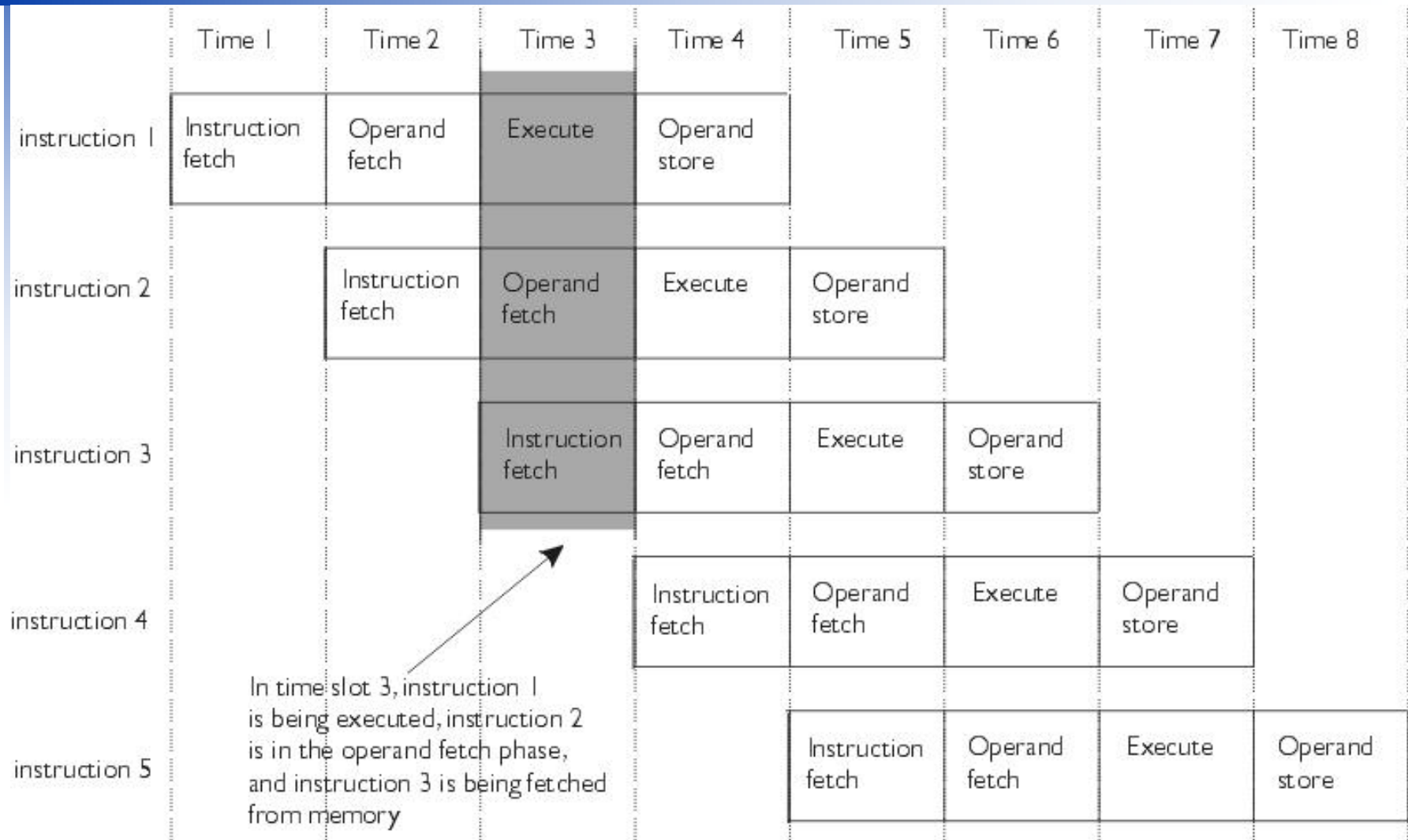
Great Idea #2: *Hierarchy of memories*



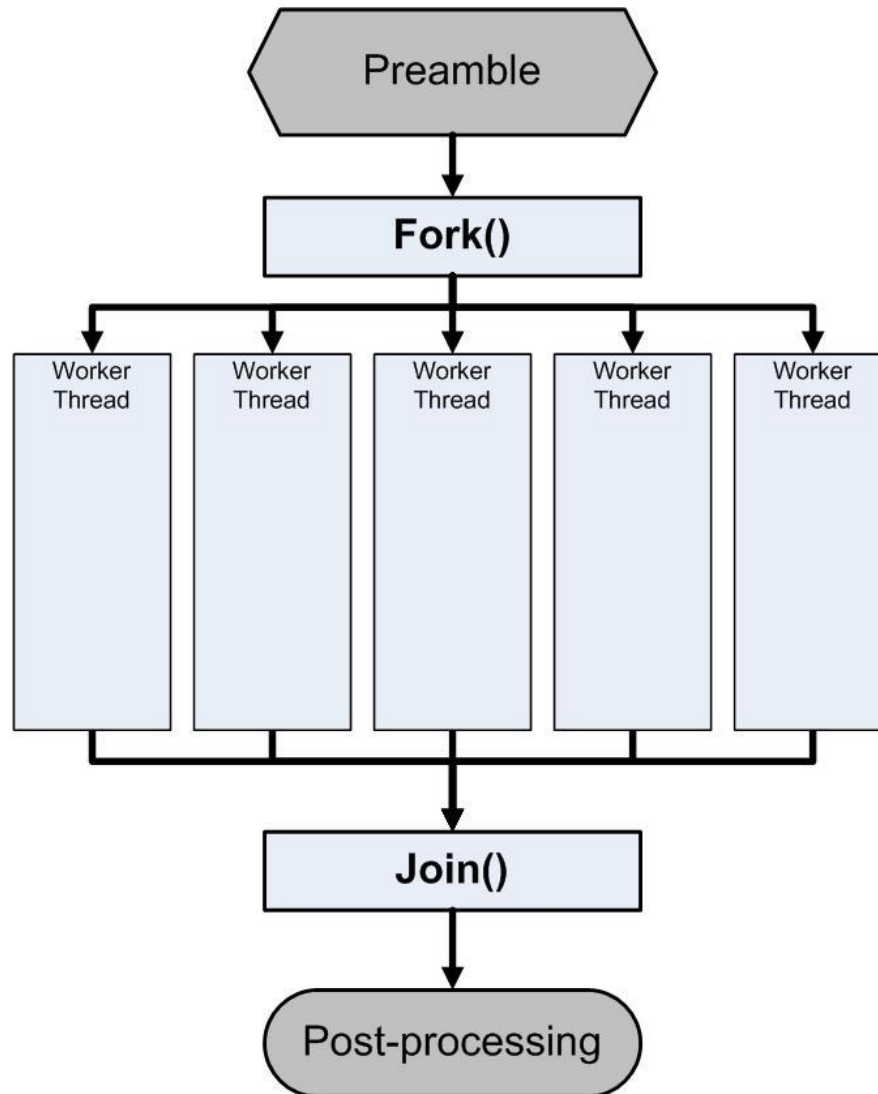
Great Idea #3: Performance *via* pipelining



Great Idea #3: Performance *via* pipelining

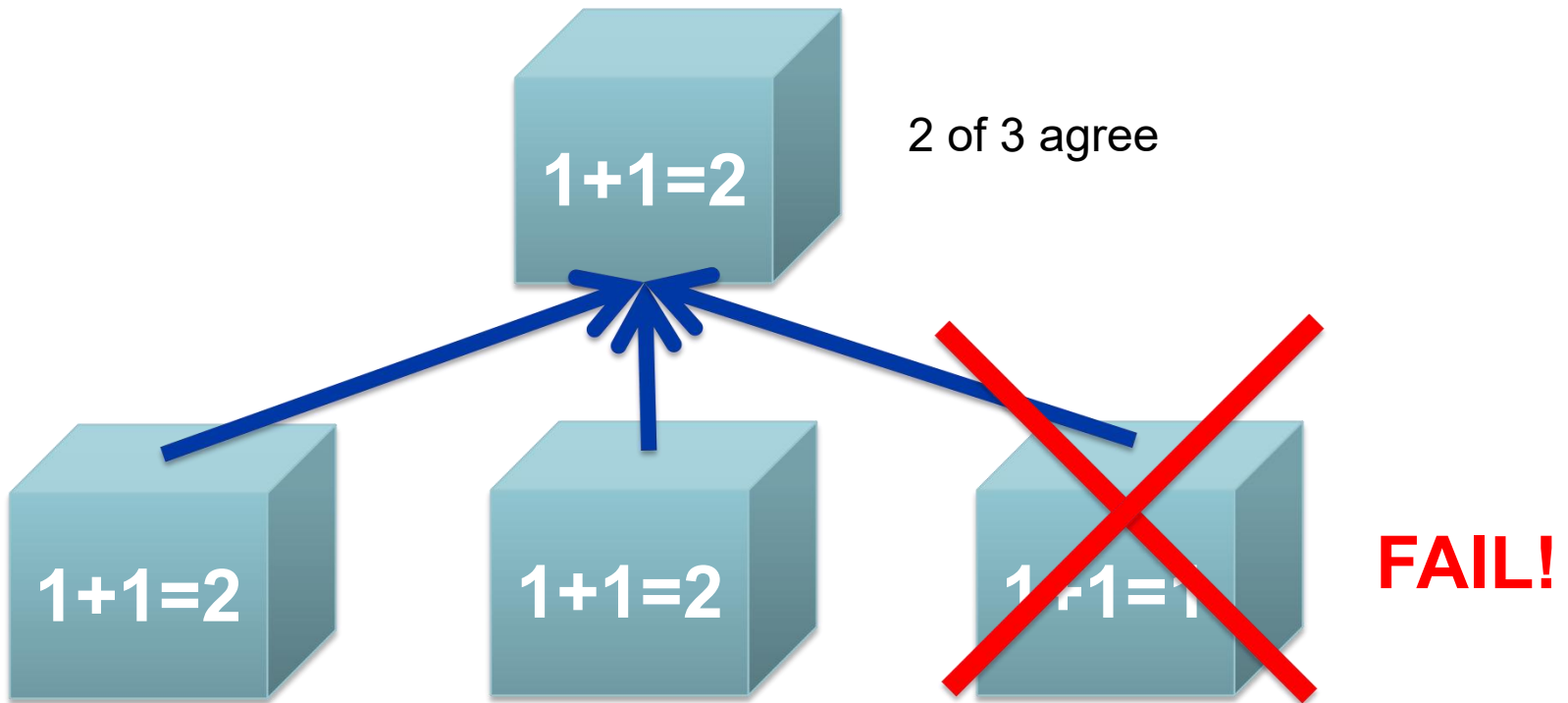


Great Idea #4: Performance *via parallelism*



Great Idea #5: *Dependability via redundancy*

- Redundancy so that a failing piece doesn't make the whole system fail



Increasing transistor density reduces the cost of redundancy

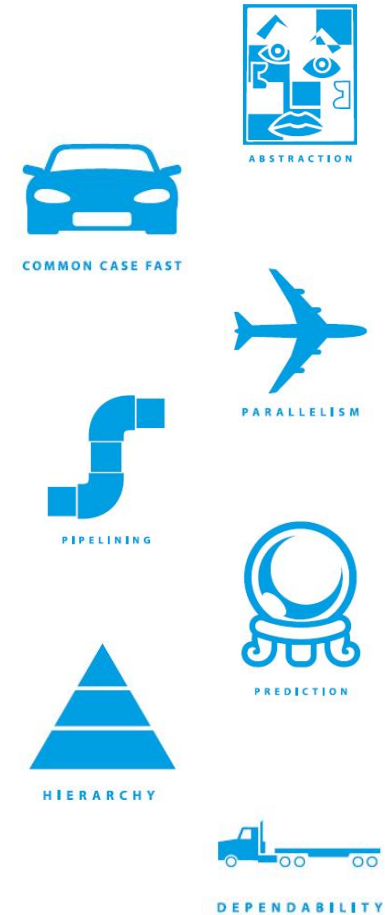
Great Idea #5: *Dependability via redundancy*

- Applies to everything from datacenters to storage to memory to instructors
 - Redundant datacenters so that can lose 1 datacenter but Internet service stays online
 - Redundant disks so that can lose 1 disk but not lose data (Redundant Arrays of Independent Disks/RAID)
 - Redundant memory bits so that can lose 1 bit but not data (Error Correcting Code/ECC Memory)



Seven Great Ideas

- Use *abstraction* to simplify design
- *Hierarchy* of memories
- Performance via *pipelining*
- Performance via *parallelism*
- *Dependability* via redundancy
- Make the **common case fast**
- Performance via **prediction**



电1010(乙)
机上收到时
间和代号

中华人民共和国邮电部

来(轉)报纸(大型)

电报

流水号	报类	发报局名	原报号数	字	数	期	时	间
X154	7	XIAN	Y0361	214	11	74		
备注								

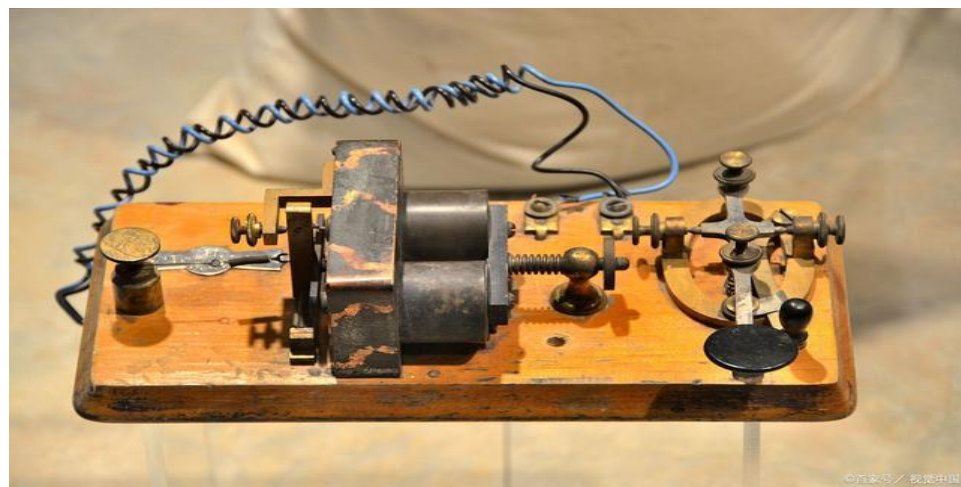
5663 盖 3944 同 4925 盖 4102 同 6203 货 0361 40674 同 LAN TIAN

3634 为 4822 号 4920 号 0361 号 0674 号 7112 号 20443 号 0664 号 7020 号 4104 号
4842 号 3810 号 2085 号 3284 号 0214 号 2624 号 2426 号 6153 号 0060 号 7711 号
7717 号 2450 号 0067 号 0061 号 1022 号 172 号 0007 号 5820 号 5887 号 2624 号
2426 号 7780 号 0001 号 3172 号 0381 号 3172 号 3788 号 3029 号 0021 号 1588 号
0511 号 0155 号 7777 号 3676 号 0446 号 2501 号 0742 号 2008 号 2651 号 753 号
0581 号 1807 号 2116 号 3172 号 1331 号 3172 号 3728 号 0031 号 1588 号 2450 号
5511 号 0155 号 2226 号 0686 号 2450 号 7112 号 3008 号 4004 号 0714 号 0664 号
4104 号 5887 号 0520 号 1906 号 0400 号 3722 号 0520 号 0007 号 0723 号 170 号
0361 号 0712 号 1417 号 0341 号 0651 号 4104 号 0820 号 0442 号 0007 号 0086 号
4842 号 3781 号 0760 号 0732 号 1201 号 4687 号 5002 号 1842 号 6014 号 5222 号

如有查询请打电话，
或随带此纸到局接洽。

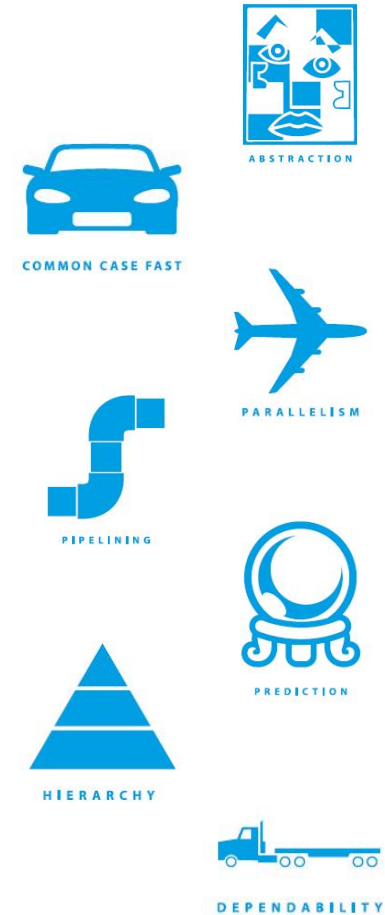
7400 =

7788收藏@S49526-2021.01.29



Seven Great Ideas

- Use *abstraction* to simplify design
- *Hierarchy* of memories
- Performance via *pipelining*
- Performance via *parallelism*
- *Dependability* via redundancy
- Make the common case fast
- Performance via ***prediction***



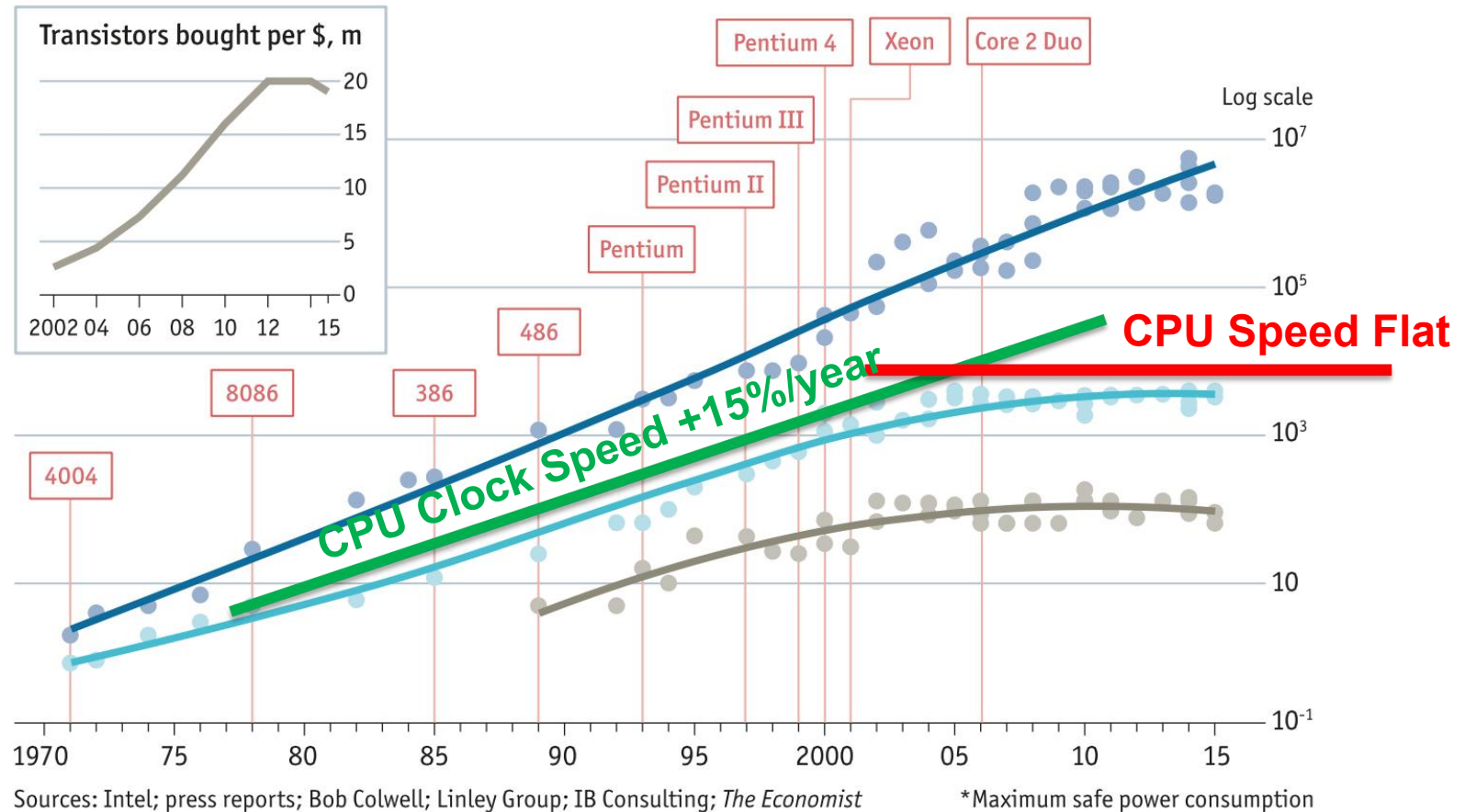
- In some cases, it can be faster on average to guess and start working rather than wait until you know for sure, assuming that the mechanism to recover from a misprediction is not too expensive and your prediction is relatively accurate.

```
if(i<100)  
    j++;
```

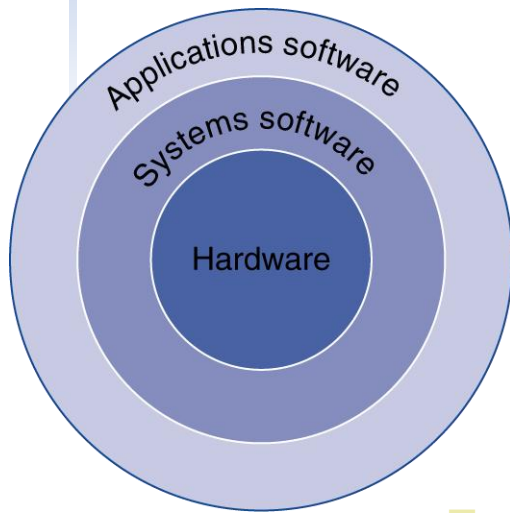
Why is Architecture Exciting Today?

Stuttering

● Transistors per chip, '000 ● Clock speed (max), MHz ● Thermal design power*, w □ Chip introduction dates, selected



Below Your Program



- **Application software**
 - Written in high-level language
- **System software**
 - **Compiler**: translates HLL code to machine code
 - **Operating System**: service code
 - Handling input/output
 - Managing memory and storage
 - Scheduling tasks & sharing resources
- **Hardware**
 - Processor(registers/datapath/control/cache), memory(主存), I/O controllers(I/O接口)

Levels of Program Code

- High-level language
 - Level of abstraction closer to problem domain
 - Provides for productivity and portability
- Assembly language
 - Textual representation of instructions
- Hardware representation
 - Binary digits (bits)
 - Encoded instructions and data

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for MIPS)

```
swap:
    muli $2, $5, 4
    add  $2, $4, $2
    lw   $15, 0($2)
    lw   $16, 4($2)
    sw   $16, 0($2)
    sw   $15, 4($2)
    jr   $31
```

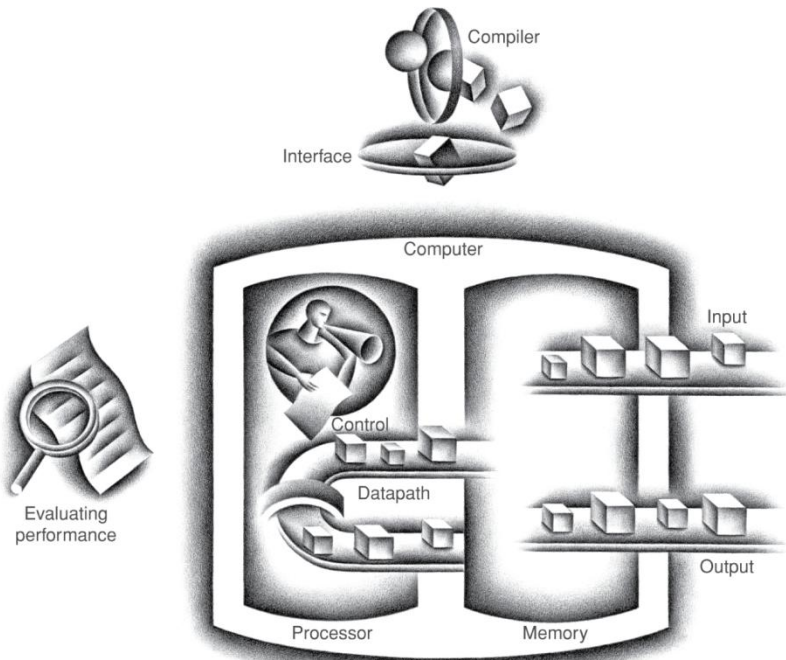
Assembler

Binary machine
language
program
(for MIPS)

```
000000001010000100000000000011000
000000000000110000001100000100001
100011000110001000000000000000000
100011001111001000000000000000100
101011001111001000000000000000000
101011000110001000000000000000100
00000011111000000000000000001000
```

Components of a Computer

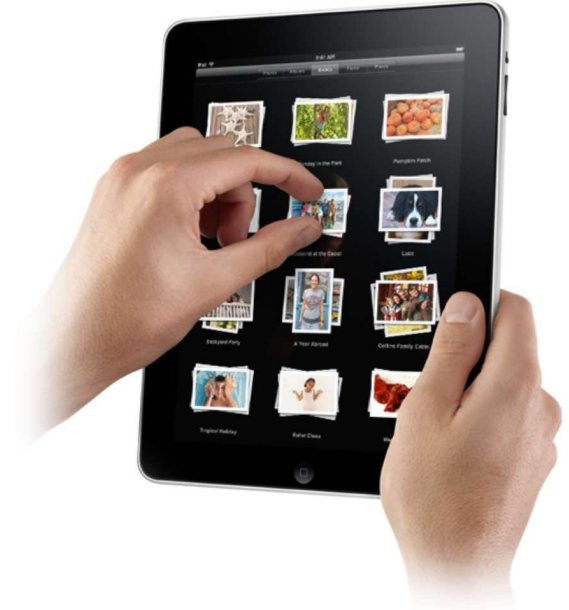
The BIG Picture



- Same components for all kinds of computer
 - Desktop, server, embedded
- Input/output includes
 - User-interface devices
 - Display, keyboard, mouse
 - **Storage devices**
 - Hard disk, CD/DVD, flash
 - Network adapters
 - For communicating with other computers

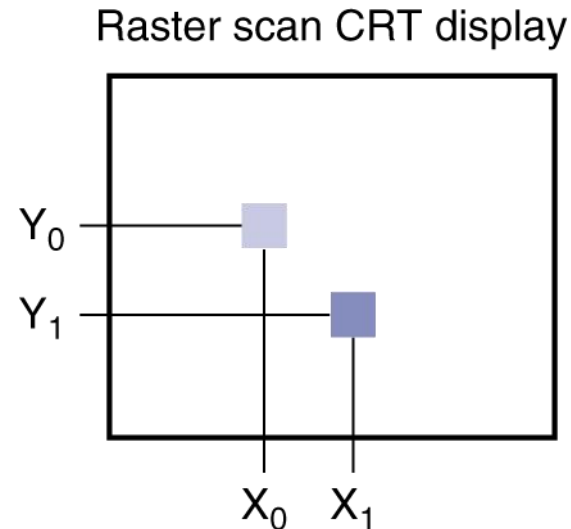
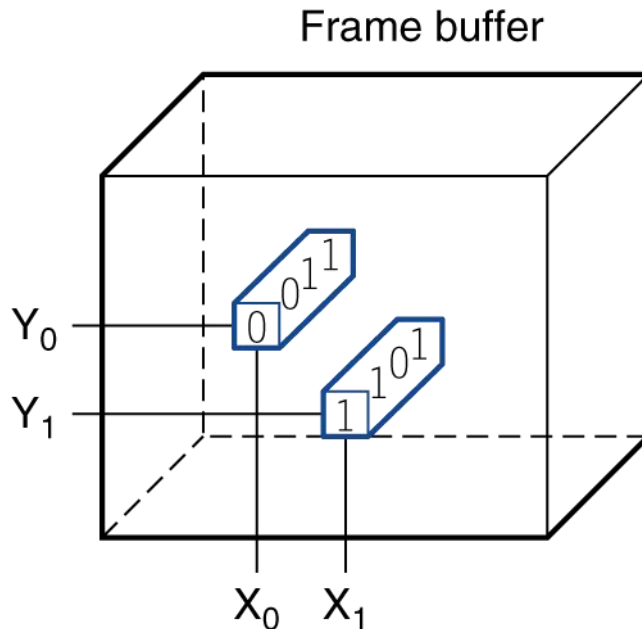
Touchscreen

- PostPC device
- Supersedes(取代) keyboard and mouse
- Resistive and Capacitive types
 - Most tablets, smart phones use capacitive
 - Capacitive allows multiple touches simultaneously

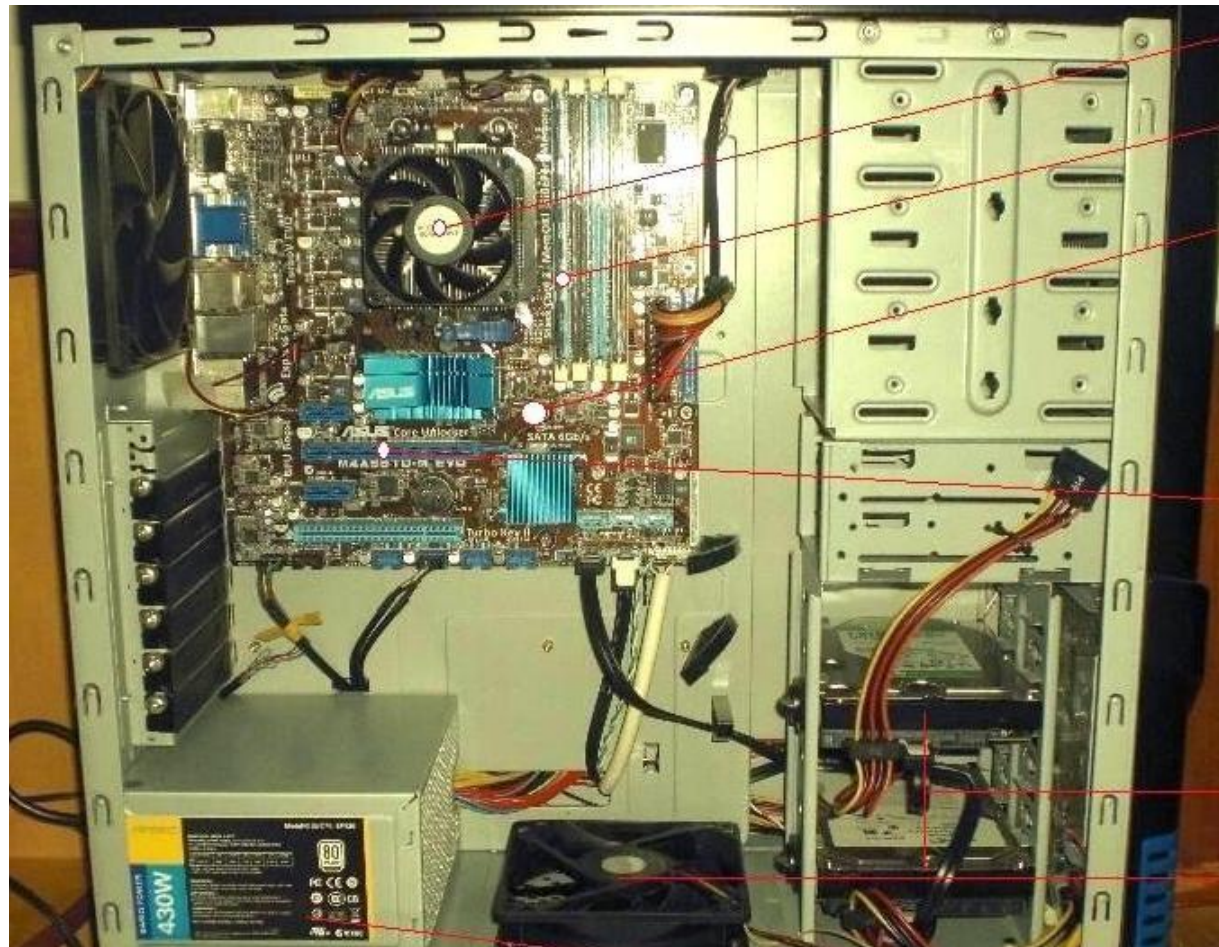


Through the Looking Glass

- LCD screen: picture elements (**pixels**)
 - Mirrors content of frame buffer memory



Opening the Box-PC



cpu散热器 压在下面的就是CPU了

CPU附近插着的是内存条

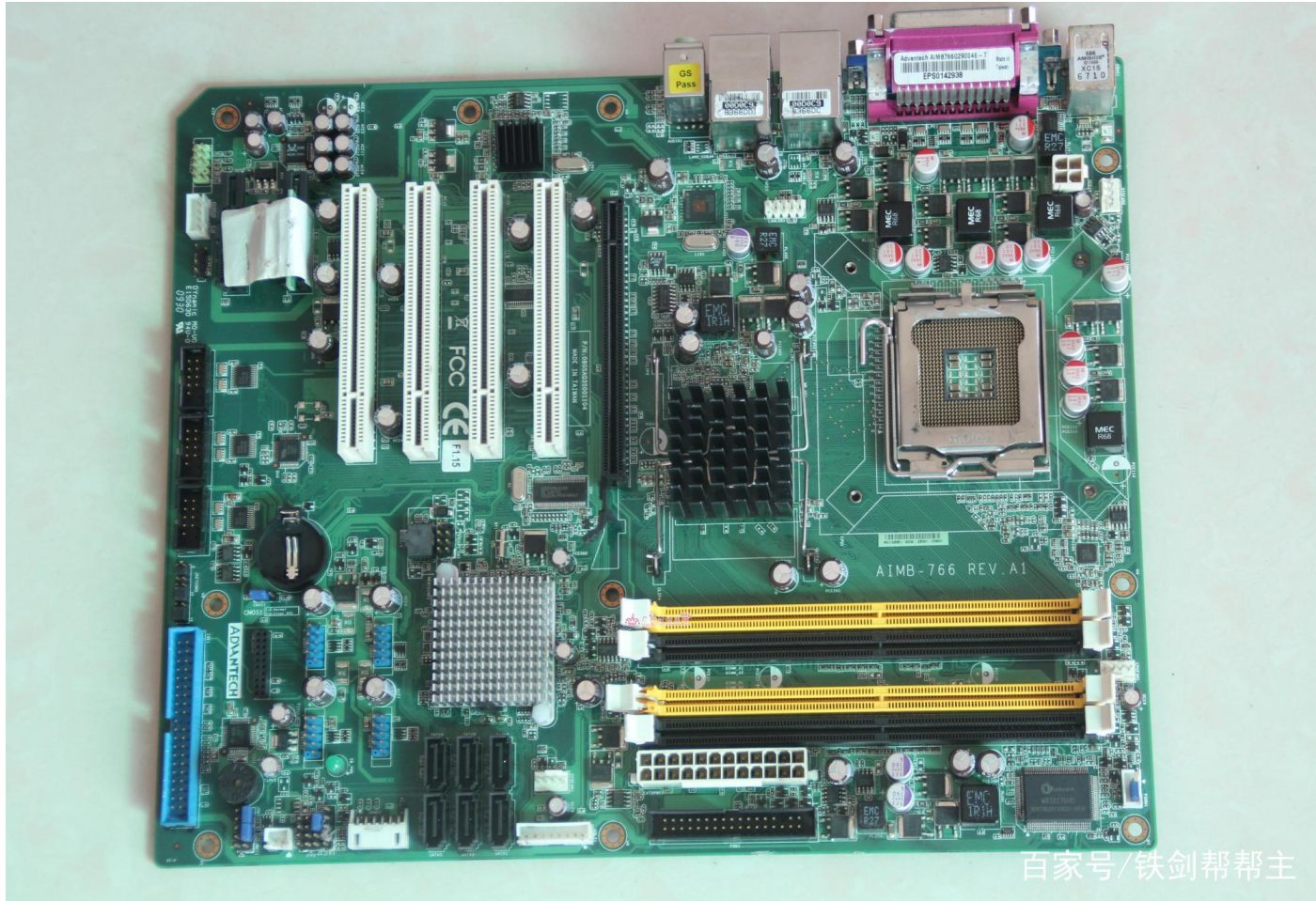
安装CPU 内存的这块板子就是主板了

用来安装显卡的显卡插槽 我的机器用的是集成显卡 这里就没有装独立显卡了

硬盘

机箱风扇

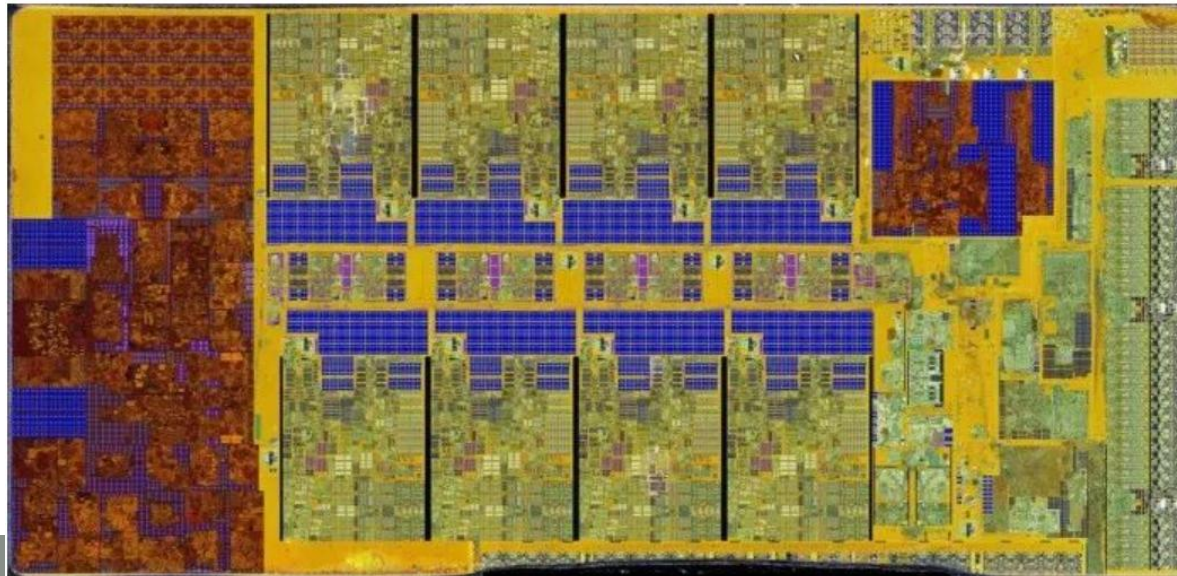
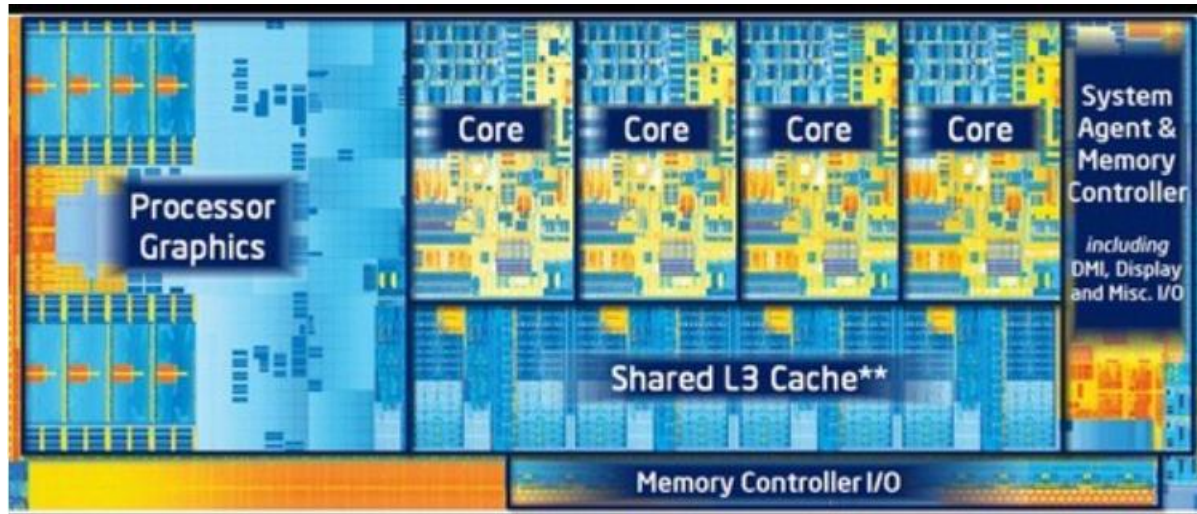
Opening the Box-PC



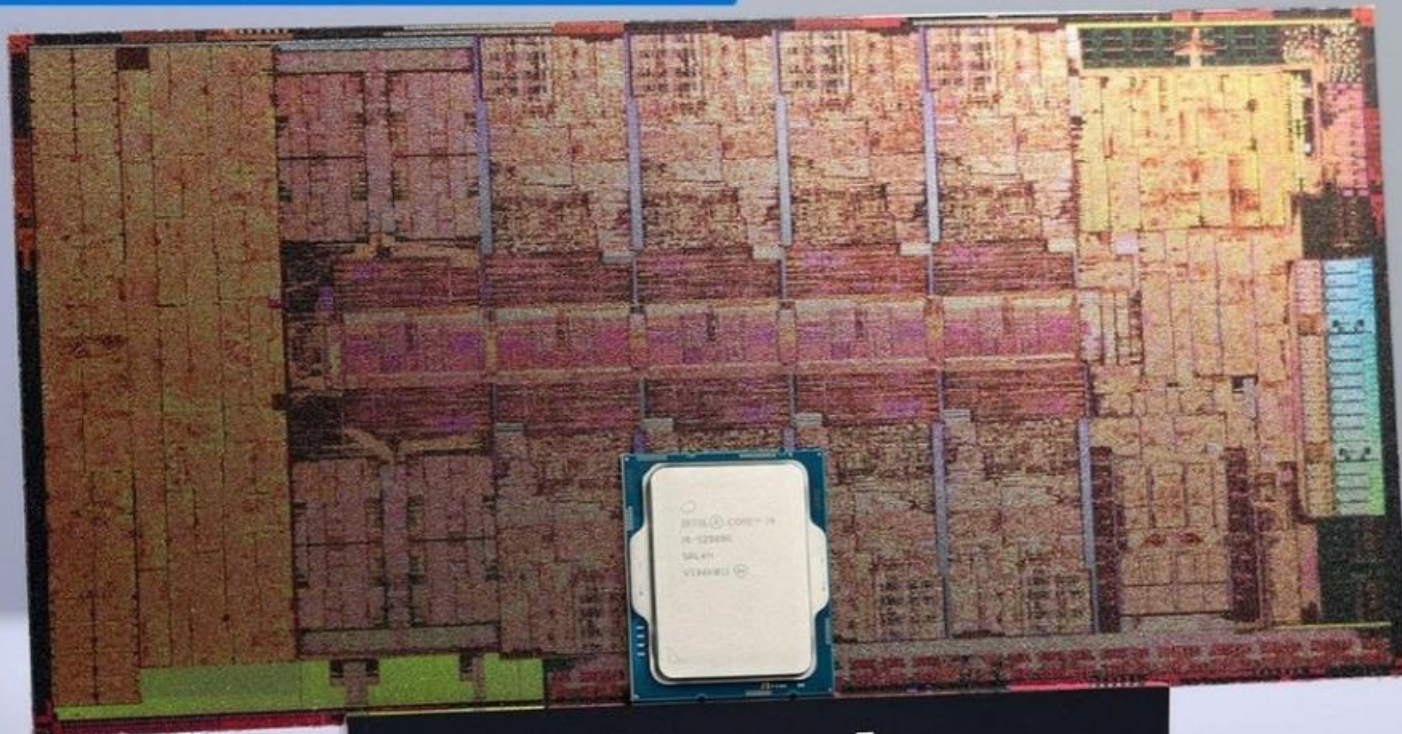
Opening the Box-PC



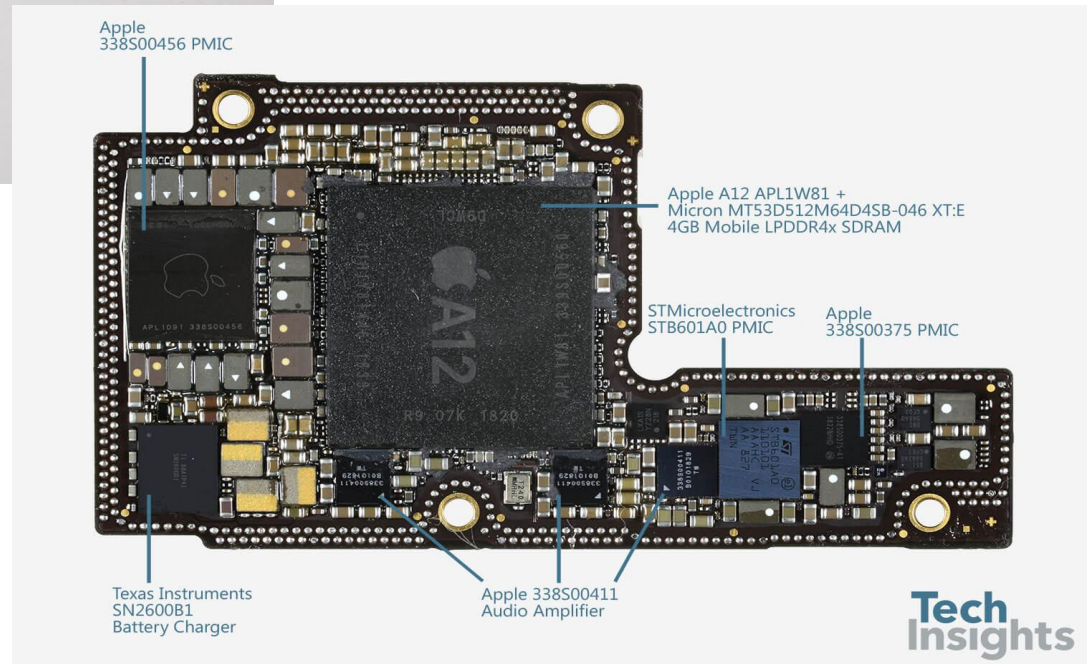
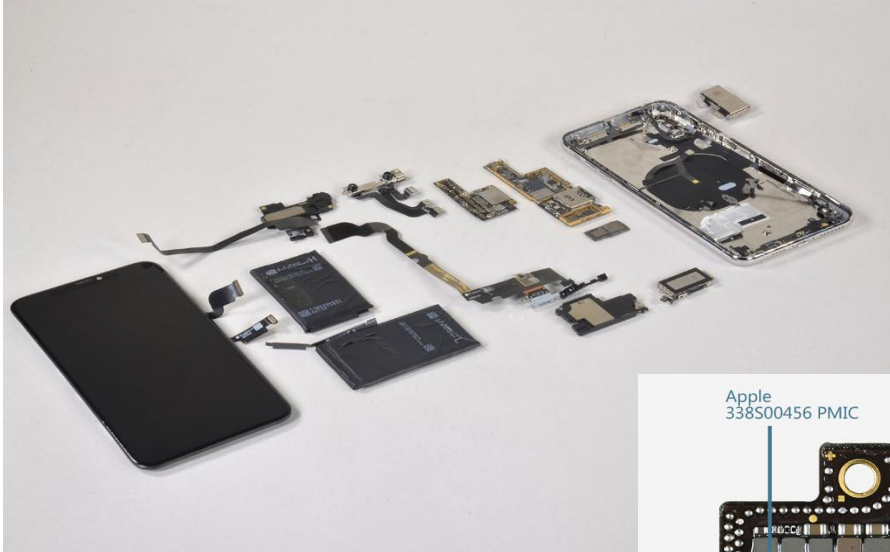
Inside the Processor



Intel Core i9-12900K@Z690



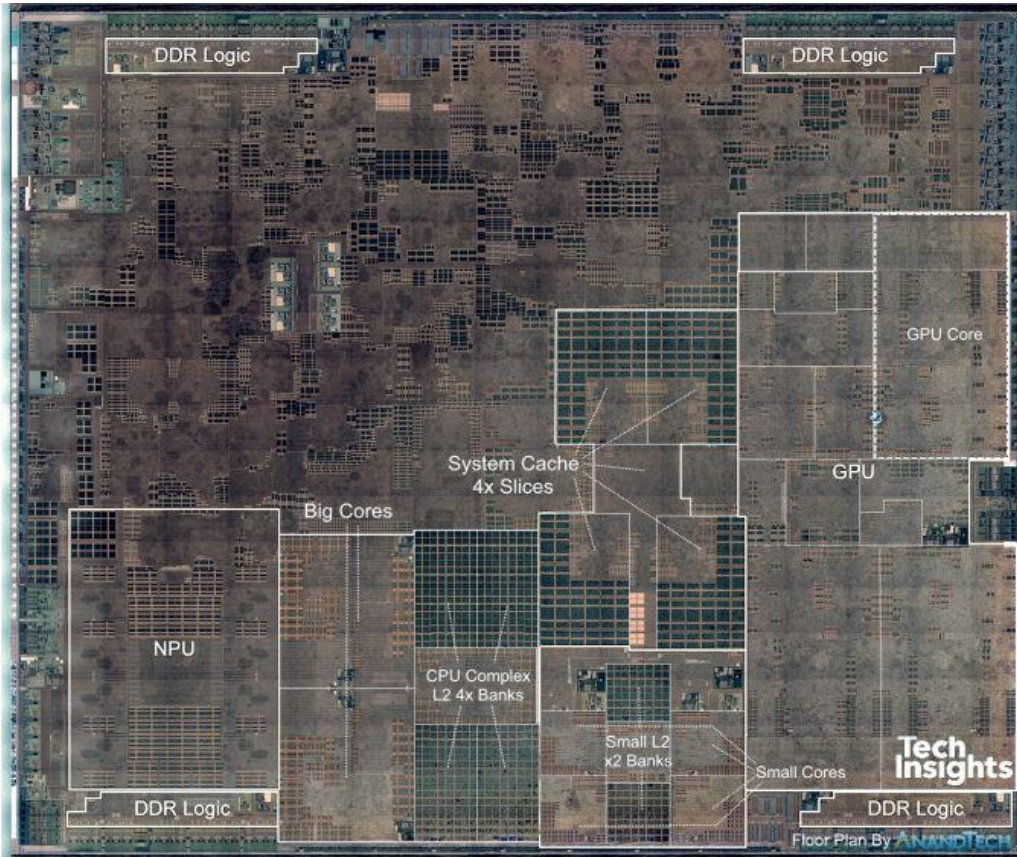
Opening the Box-Smart phone



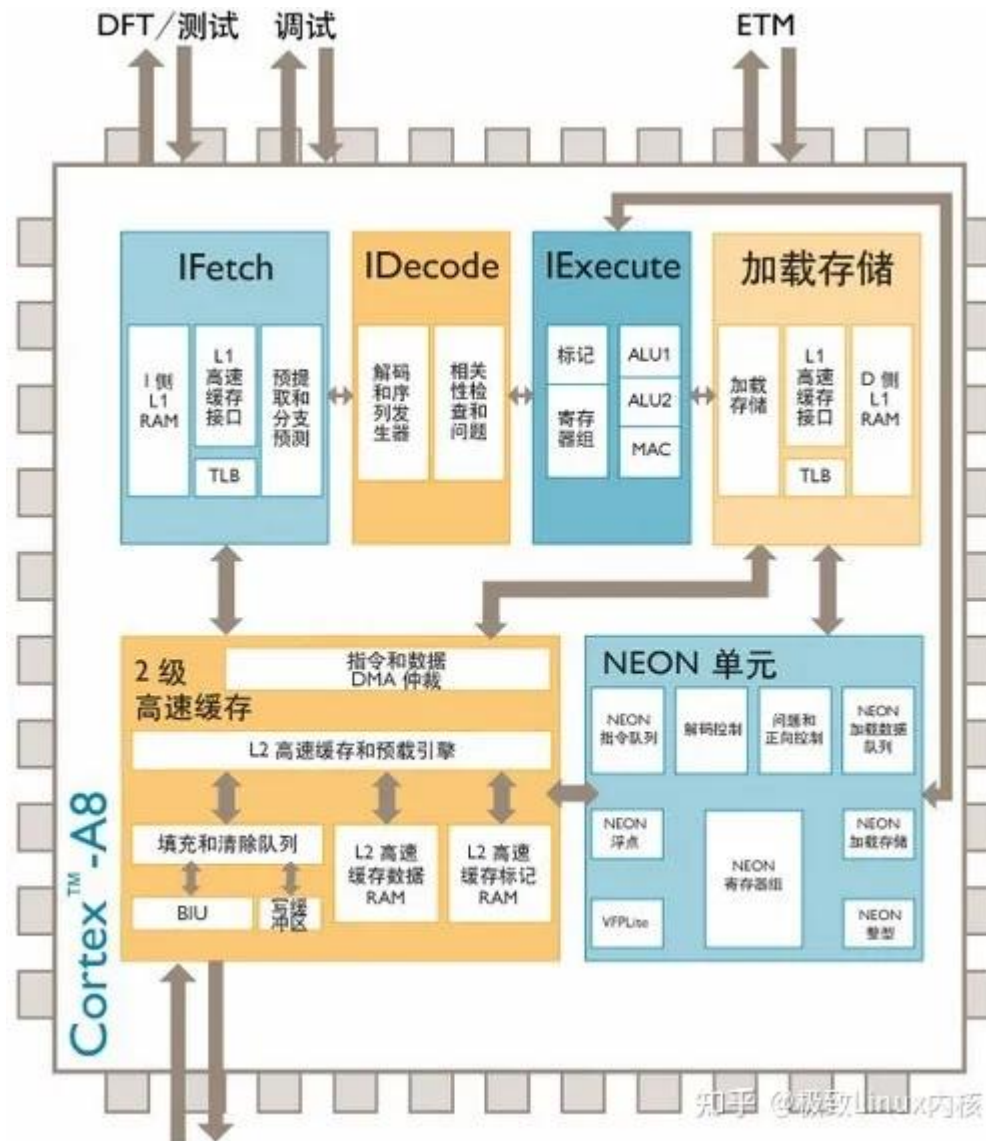
Tech
Insights

Inside the Processor

- A12 processor



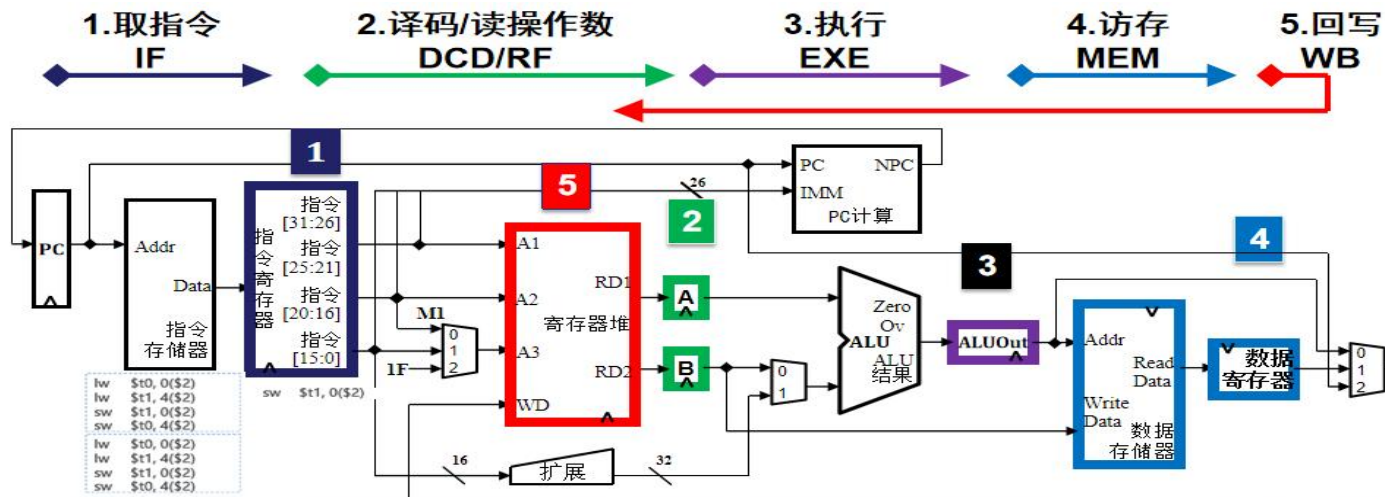
Inside the Processor



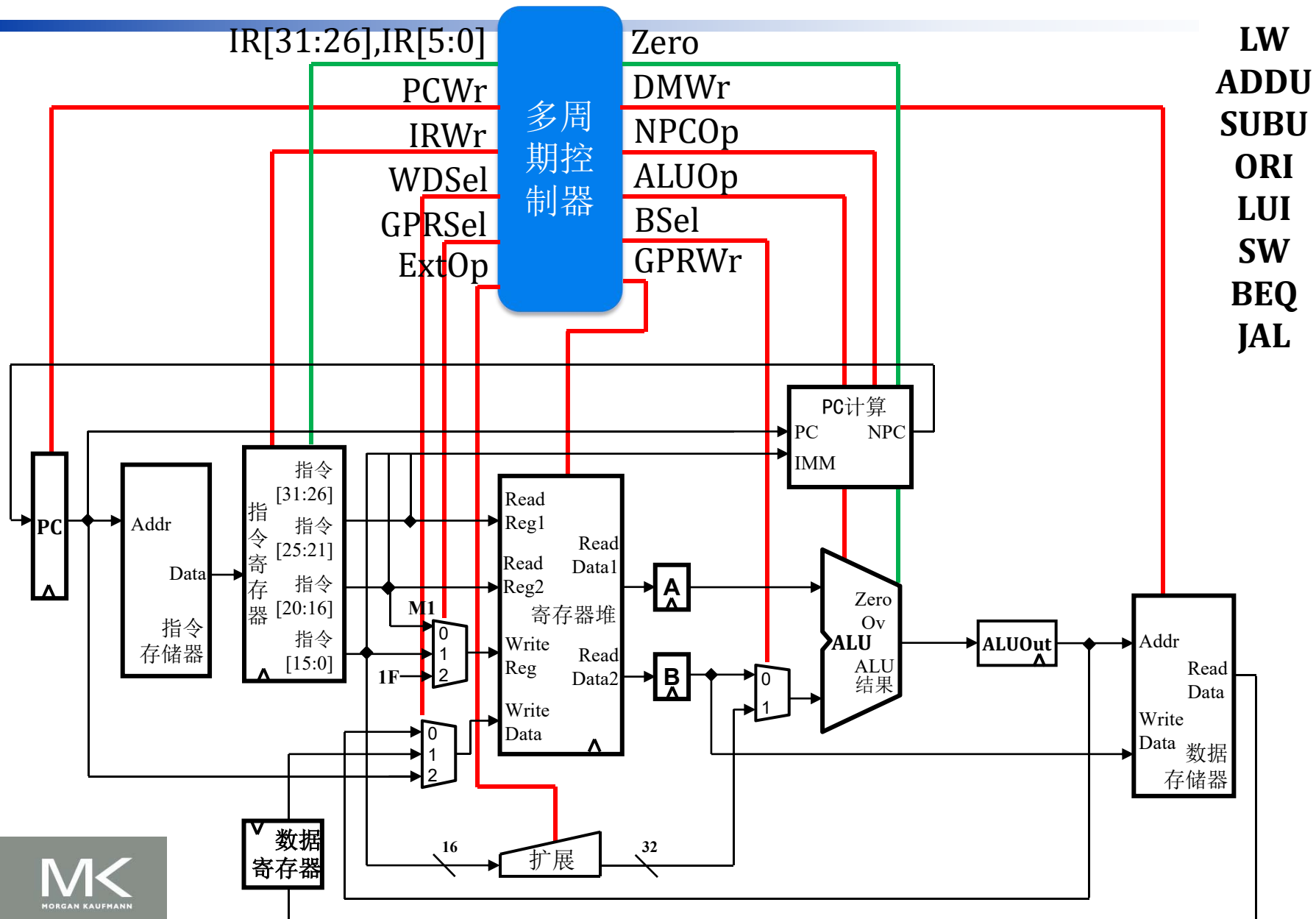
知乎 @极效linux内核

Inside the Processor (CPU)

- **Datapath**: performs operations on data
- **Control**: sequences datapath, memory, ...
- **Cache memory**
 - Small fast SRAM memory for immediate access to data



Datapath and control example



Abstractions

The BIG Picture

- Abstraction helps us deal with complexity
 - Hide lower-level detail
- Instruction set architecture (ISA)
 - The hardware/software interface
- Application binary interface
 - The ISA plus system software interface
- Implementation
 - The details underlying and interface



23级数媒-计算机组成...

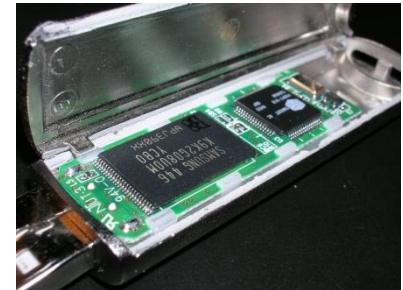
群号: 957988725



扫一扫二维码，加入群聊

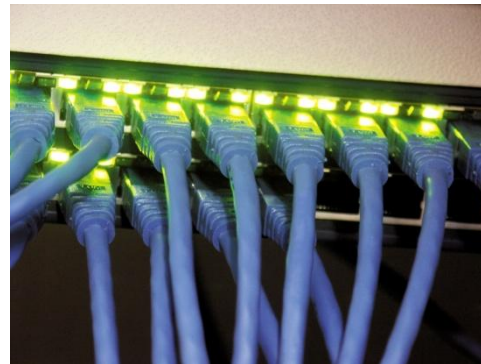
A Safe Place for Data

- Volatile main memory
 - Loses instructions and data when power off
- Non-volatile secondary memory
 - Magnetic disk
 - Flash memory
 - Optical disk (CDROM, DVD)



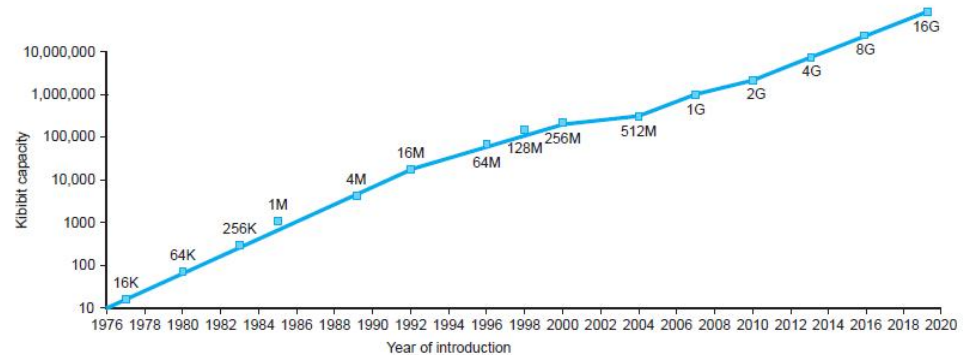
Networks

- Communication, resource sharing, nonlocal access
- Local area network (LAN): Ethernet
- Wide area network (WAN): the Internet
- Wireless network: WiFi, Bluetooth



Technology Trends

- Electronics technology continues to evolve
 - Increased capacity and performance
 - Reduced cost



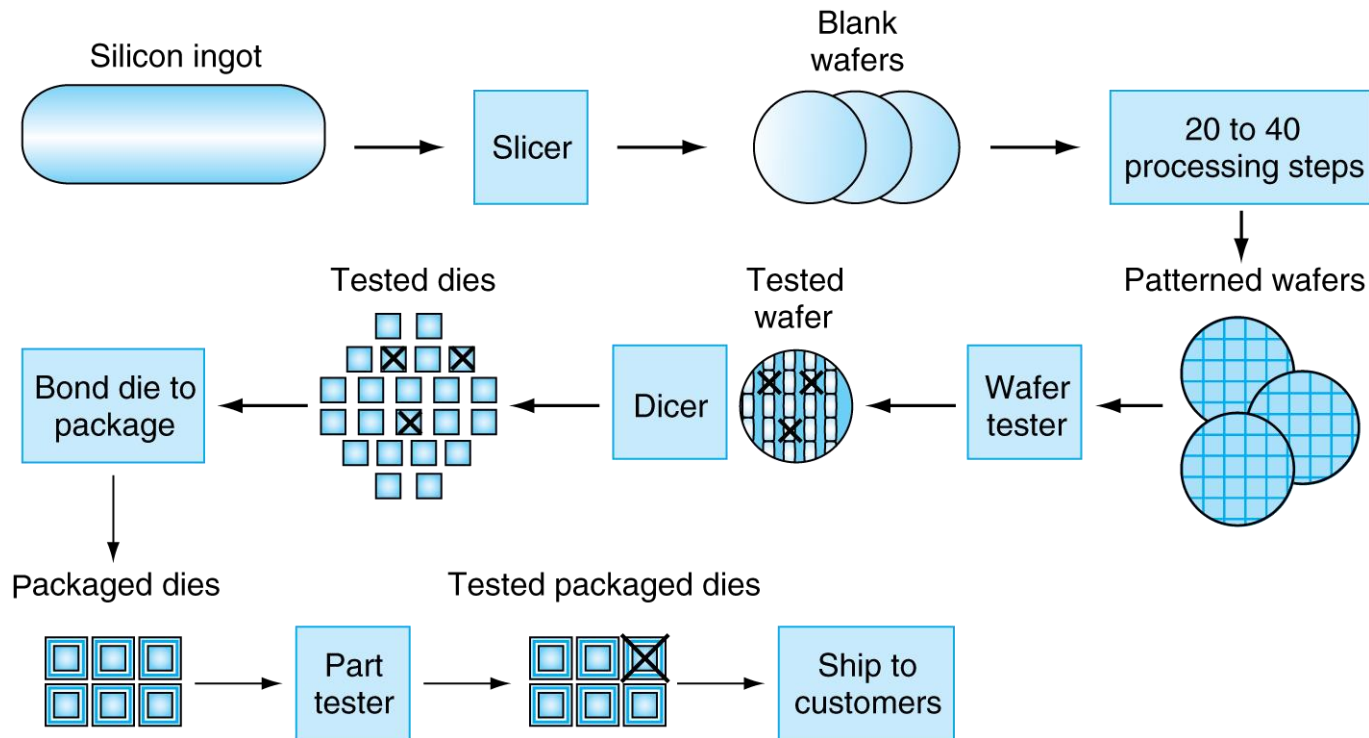
DRAM capacity

Year	Technology	Relative performance/cost
1951	Vacuum tube	1
1965	Transistor	35
1975	Integrated circuit (IC)	900
1995	Very large scale IC (VLSI)	2,400,000
2013	Ultra large scale IC	250,000,000,000

Semiconductor Technology

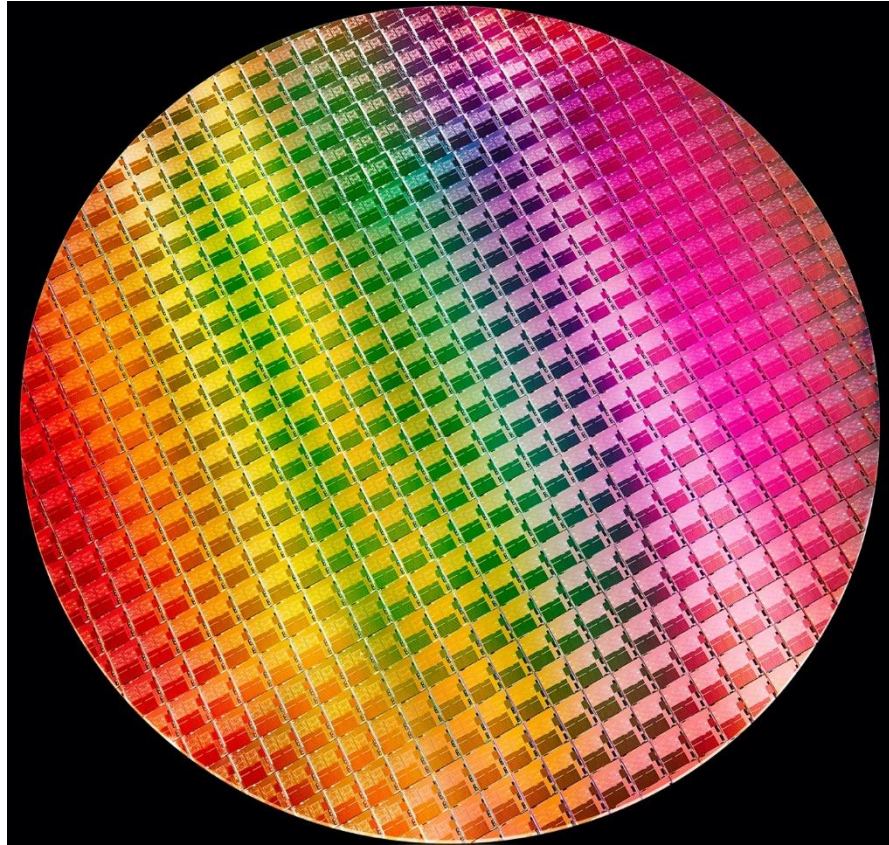
- Silicon: semiconductor
- Add materials to transform properties:
 - Conductors
 - Insulators
 - Switch

Manufacturing ICs



- **Yield:** proportion of working dies per wafer

Intel® Core 10th Gen



- 300mm wafer, 506 chips, 10nm technology
- Each chip is 11.4 x 10.7 mm

Integrated Circuit Cost

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

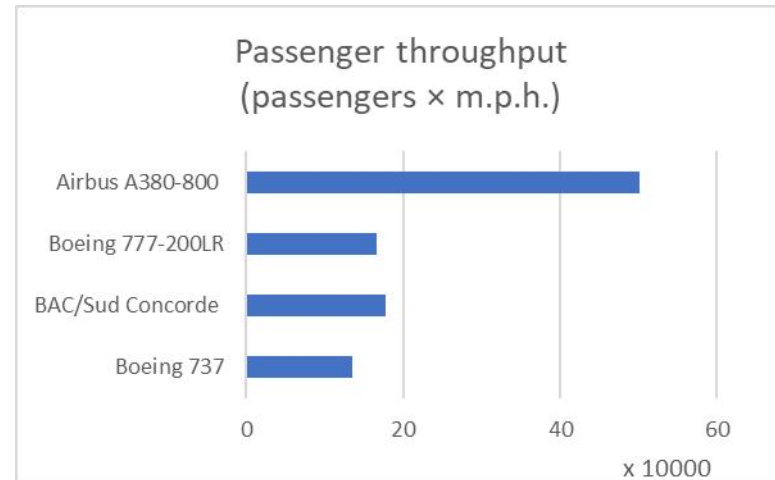
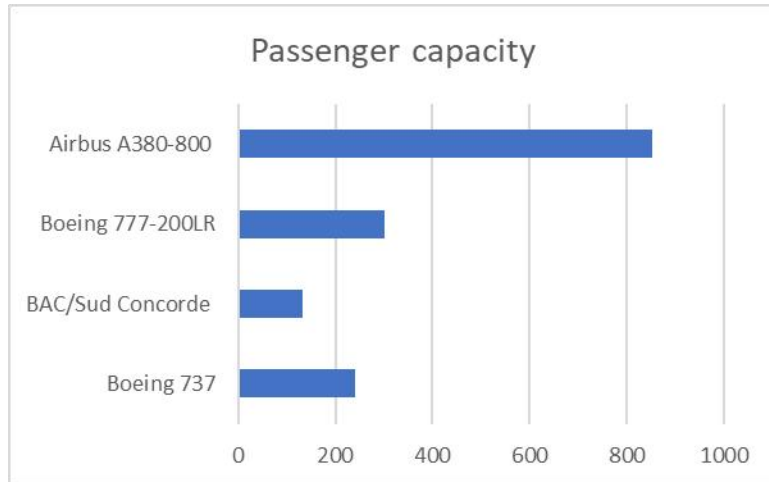
$$\text{Dies per wafer} \approx \text{Wafer area} / \text{Die area}$$

$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area} / 2))^2}$$

- Nonlinear relation to area and defect rate
 - Wafer cost and area are fixed
 - Defect rate determined by manufacturing process
 - Die area determined by architecture and circuit design

Defining Performance

- Which airplane has the best performance?



Response Time and Throughput

- **Response time** (响应时间)
 - How long it takes to do a task
- **Throughput** (吞吐率)
 - Total work done per unit time
 - e.g., tasks/transactions/... per hour
- How are response time and throughput affected by
 - Replacing the processor with a faster version?
 - Adding more processors?
- We'll focus on response time for now...

Relative Performance

- Define Performance = 1/Execution Time
- “X is n time faster than Y”

$$\begin{aligned} & \text{Performance}_X / \text{Performance}_Y \\ &= \text{Execution time}_Y / \text{Execution time}_X = n \end{aligned}$$

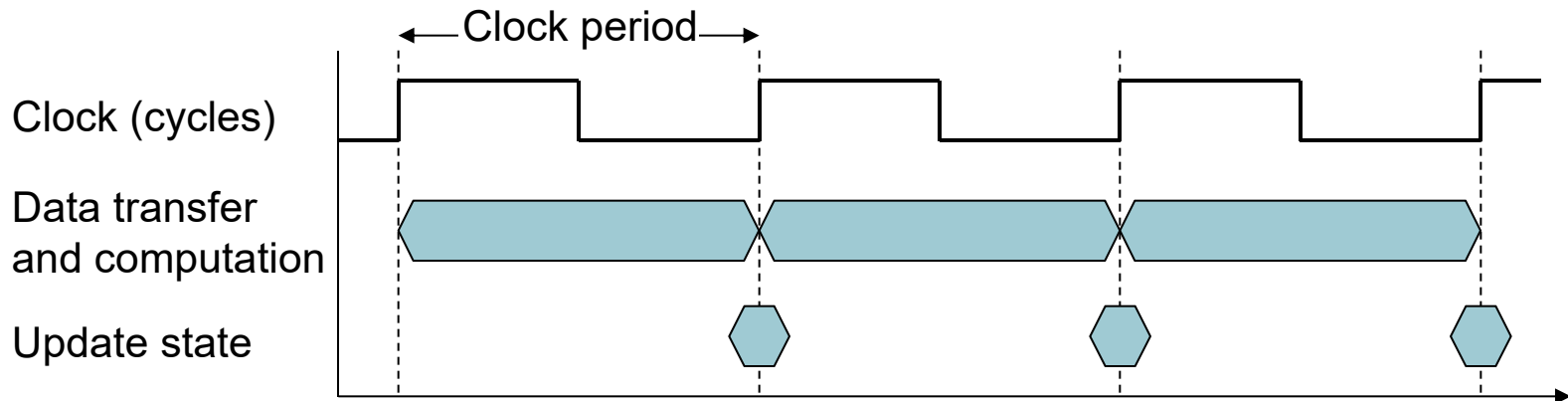
- Example: time taken to run a program
 - 10s on A, 15s on B
 - $\text{Execution Time}_B / \text{Execution Time}_A$
 $= 15\text{s} / 10\text{s} = 1.5$
 - So A is 1.5 times faster than B

Measuring Execution Time

- Elapsed time
 - Total response time, including all aspects
 - Processing, I/O, OS overhead, idle time
 - Determines system performance
- **CPU time (CPU时间)**
 - Time spent processing a given job
 - Discounts I/O time, other jobs' shares
 - Comprises **user CPU time** and **system CPU time**
 - Different programs are affected differently by CPU and system performance

CPU Clocking

- Operation of digital hardware governed by a constant-rate clock



- Clock period: duration of a clock cycle
 - e.g., $250\text{ps} = 0.25\text{ns} = 250 \times 10^{-12}\text{s}$
- Clock frequency (rate): cycles per second
 - e.g., $4.0\text{GHz} = 4000\text{MHz} = 4.0 \times 10^9\text{Hz}$

CPU Time

$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Cycle Time} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}\end{aligned}$$

- Performance improved by
 - Reducing number of clock cycles
 - Increasing clock rate
 - Hardware designer must often trade off clock rate against cycle count

CPU Time Example

- Computer A: 2GHz clock, 10s CPU time
- Designing Computer B
 - Aim for 6s CPU time
 - Can do faster clock, but causes $1.2 \times$ clock cycles
- How fast must Computer B clock be?

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6s}$$

$$\begin{aligned}\text{Clock Cycles}_A &= \text{CPU Time}_A \times \text{Clock Rate}_A \\ &= 10s \times 2\text{GHz} = 20 \times 10^9\end{aligned}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6s} = \frac{24 \times 10^9}{6s} = 4\text{GHz}$$

Instruction Count and CPI

Clock Cycles = Instruction Count $\times$ Cycles per Instruction

CPU Time = Instruction Count $\times$ CPI $\times$ Clock Cycle Time

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

- Instruction Count for a program
 - Determined by program, ISA and compiler
- Average cycles per instruction
 - Determined by CPU hardware
 - If different instructions have different CPI
 - Average CPI affected by instruction mix

CPI Example

- Computer A: Cycle Time = 250ps, CPI = 2.0
- Computer B: Cycle Time = 500ps, CPI = 1.2
- Same ISA
- Which is faster, and by how much?

$$\begin{aligned}\text{CPU Time}_A &= \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A \\ &= 1 \times 2.0 \times 250\text{ps} = 1 \times 500\text{ps} \leftarrow \text{A is faster...}\end{aligned}$$

$$\begin{aligned}\text{CPU Time}_B &= \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B \\ &= 1 \times 1.2 \times 500\text{ps} = 1 \times 600\text{ps}\end{aligned}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{1 \times 600\text{ps}}{1 \times 500\text{ps}} = 1.2 \leftarrow \text{...by this much}$$

CPI in More Detail

- If different instruction classes take different numbers of cycles

$$\text{Clock Cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Instruction Count}_i)$$

- Weighted average CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \frac{\text{Instruction Count}_i}{\text{Instruction Count}} \right)$$

Relative frequency

CPI Example

- Alternative compiled code sequences using instructions in classes A, B, C

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

- Sequence 1: IC = 5

- Clock Cycles
 $= 2 \times 1 + 1 \times 2 + 2 \times 3$
 $= 10$
- Avg. CPI = $10/5 = 2.0$

- Sequence 2: IC = 6

- Clock Cycles
 $= 4 \times 1 + 1 \times 2 + 1 \times 3$
 $= 9$
- Avg. CPI = $9/6 = 1.5$

Performance Summary

The BIG Picture

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

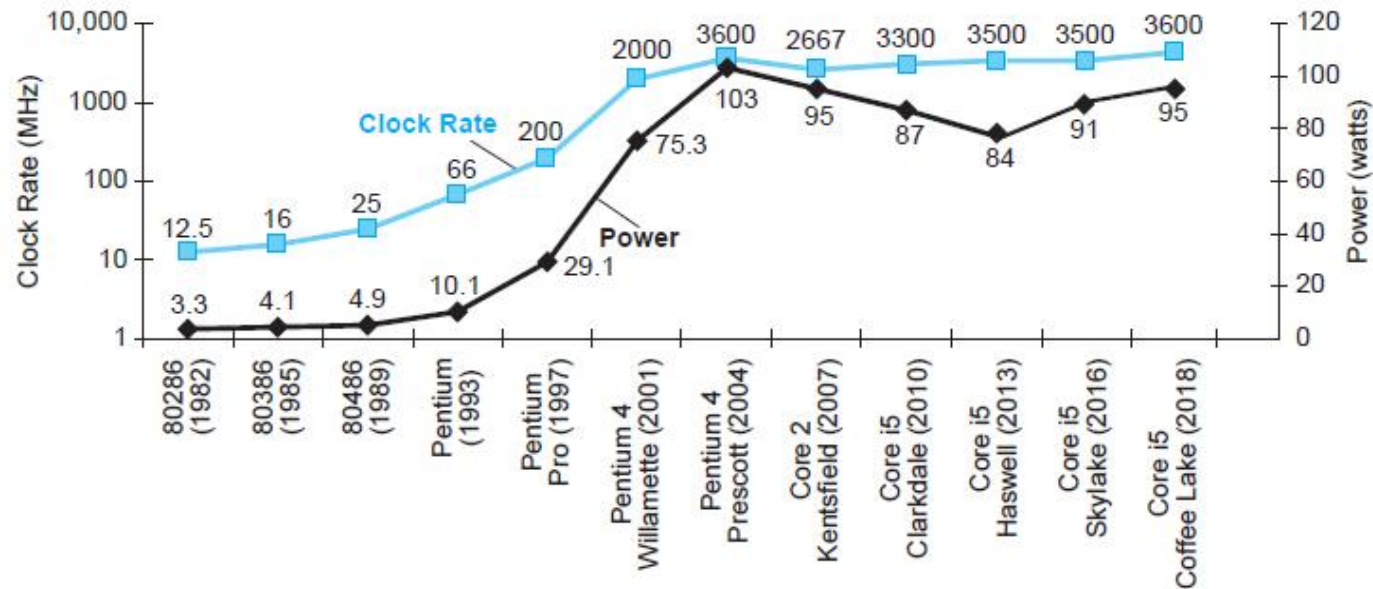
- Performance depends on
 - **Algorithm**: affects IC, possibly CPI
 - **Programming language**: affects IC, CPI
 - **Compiler**: affects IC, CPI
 - **Instruction set architecture**: affects IC, CPI, T_c

用一台时钟频率为400MHz的处理机执行如表1.2所示标准测试程序, 程序中包含的各种指令条数及各指令的平均时钟周期数如表1.2所示。求测试程序的执行时间及CPI。

表 1.2 各类指令条数和指令的平均时钟周期数

指令类型	指令条数	平均时钟周期数
整数类型	40 000	1
数据传送	30 000	2
浮点运算	20 000	2
控制传送	10 000	2

Power Trends



- In CMOS IC technology

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

×30

5V → 1V

×1000

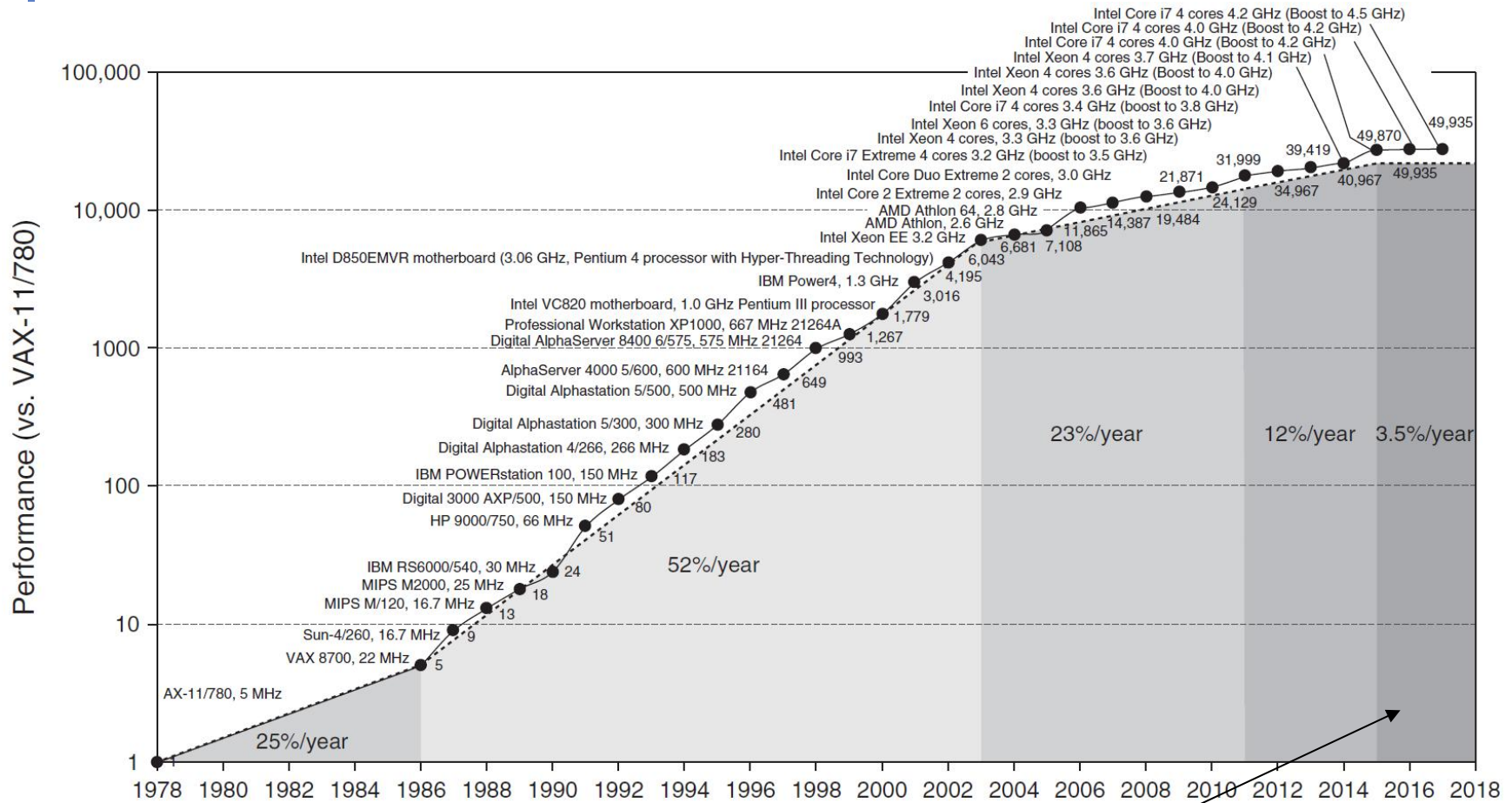
Reducing Power

- Suppose a new CPU has
 - 85% of capacitive load of old CPU
 - 15% voltage and 15% frequency reduction

$$\frac{P_{\text{new}}}{P_{\text{old}}} = \frac{C_{\text{old}} \times 0.85 \times (V_{\text{old}} \times 0.85)^2 \times F_{\text{old}} \times 0.85}{C_{\text{old}} \times V_{\text{old}}^2 \times F_{\text{old}}} = 0.85^4 = 0.52$$

- The power wall
 - We can't reduce voltage further
 - We can't remove more heat
- How else can we improve performance?

Uniprocessor Performance



Constrained by power, instruction-level parallelism, memory latency

Multiprocessors

- Multicore microprocessors
 - More than one processor per chip
- Requires explicitly parallel programming
 - Compare with instruction level parallelism
 - Hardware executes multiple instructions at once
 - Hidden from the programmer
 - Hard to do
 - Programming for performance
 - Load balancing
 - Optimizing communication and synchronization

SPEC CPU Benchmark

- Programs used to measure performance
 - Supposedly typical of actual workload
- Standard Performance Evaluation Corp (SPEC)
 - Develops benchmarks for CPU, I/O, Web, ...
- SPEC CPU2006
 - Elapsed time to execute a selection of programs
 - Negligible I/O, so focuses on CPU performance
 - Normalize relative to reference machine
 - Summarize as geometric mean of performance ratios
 - CINT2006 (integer) and CFP2006 (floating-point)

$$\sqrt[n]{\prod_{i=1}^n \text{Execution time ratio}_i}$$

SPECspeed 2017 Integer benchmarks on a 1.8 GHz Intel Xeon E5-2650L

<i>Description</i>	<i>Name</i>	<i>Instruction Count x 10⁹</i>	<i>CPI</i>	<i>Clock cycle time (seconds x 10⁻⁹)</i>	<i>Execution Time (seconds)</i>	<i>Reference Time (seconds)</i>	<i>SPECratio</i>
Perl interpreter	perlbench	2684	0.42	0.556	627	1774	2.83
GNU C compiler	gcc	2322	0.67	0.556	863	3976	4.61
Route planning	mcf	1786	1.22	0.556	1215	4721	3.89
Discrete Event simulation - computer network	omnetpp	1107	0.82	0.556	507	1630	3.21
XML to HTML conversion via XSLT	xalancbmk	1314	0.75	0.556	549	1417	2.58
Video compression	x264	4488	0.32	0.556	813	1763	2.17
Artificial Intelligence: alpha-beta tree search (Chess)	deepsjeng	2216	0.57	0.556	698	1432	2.05
Artificial Intelligence: Monte Carlo tree search (Go)	leela	2236	0.79	0.556	987	1703	1.73
Artificial Intelligence: recursive solution generator (Sudoku)	exchange2	6683	0.46	0.556	1718	2939	1.71
General data compression	xz	8533	1.32	0.556	6290	6182	0.98
Geometric mean							2.36

SPEC Power Benchmark

- Power consumption of server at different workload levels
 - Performance: ssj_ops/sec
 - Power: Watts (Joules/sec)

$$\text{Overall ssj_ops per Watt} = \left(\sum_{i=0}^{10} \text{ssj_ops}_i \right) / \left(\sum_{i=0}^{10} \text{power}_i \right)$$

SPECpower_ssj2008 for Xeon E5-2650L

Target Load %	Performance (ssj_ops)	Average Power (watts)
100%	4,864,136	347
90%	4,389,196	312
80%	3,905,724	278
70%	3,418,737	241
60%	2,925,811	212
50%	2,439,017	183
40%	1,951,394	160
30%	1,461,411	141
20%	974,045	128
10%	485,973	115
0%	0	48
Overall Sum	26,815,444	2,165
$\Sigma \text{ssj_ops} / \Sigma \text{power} =$		12,385

Pitfall: Amdahl's Law

- Improving an aspect of a computer and expecting a proportional improvement in overall performance

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$$

- Example: multiply accounts for 80s/100s
 - How much improvement in multiply performance to get 5× overall?

$$20 = \frac{80}{n} + 20 \quad \quad \quad \blacksquare \text{ Can't be done!}$$

- Corollary: make the common case fast

Fallacy: Low Power at Idle

- Look back at i7 power benchmark
 - At 100% load: 258W
 - At 50% load: 170W (66%)
 - At 10% load: 121W (47%)
- Google data center
 - Mostly operates at 10% – 50% load
 - At 100% load less than 1% of the time
- Consider designing processors to make power proportional to load

Pitfall: MIPS as a Performance Metric

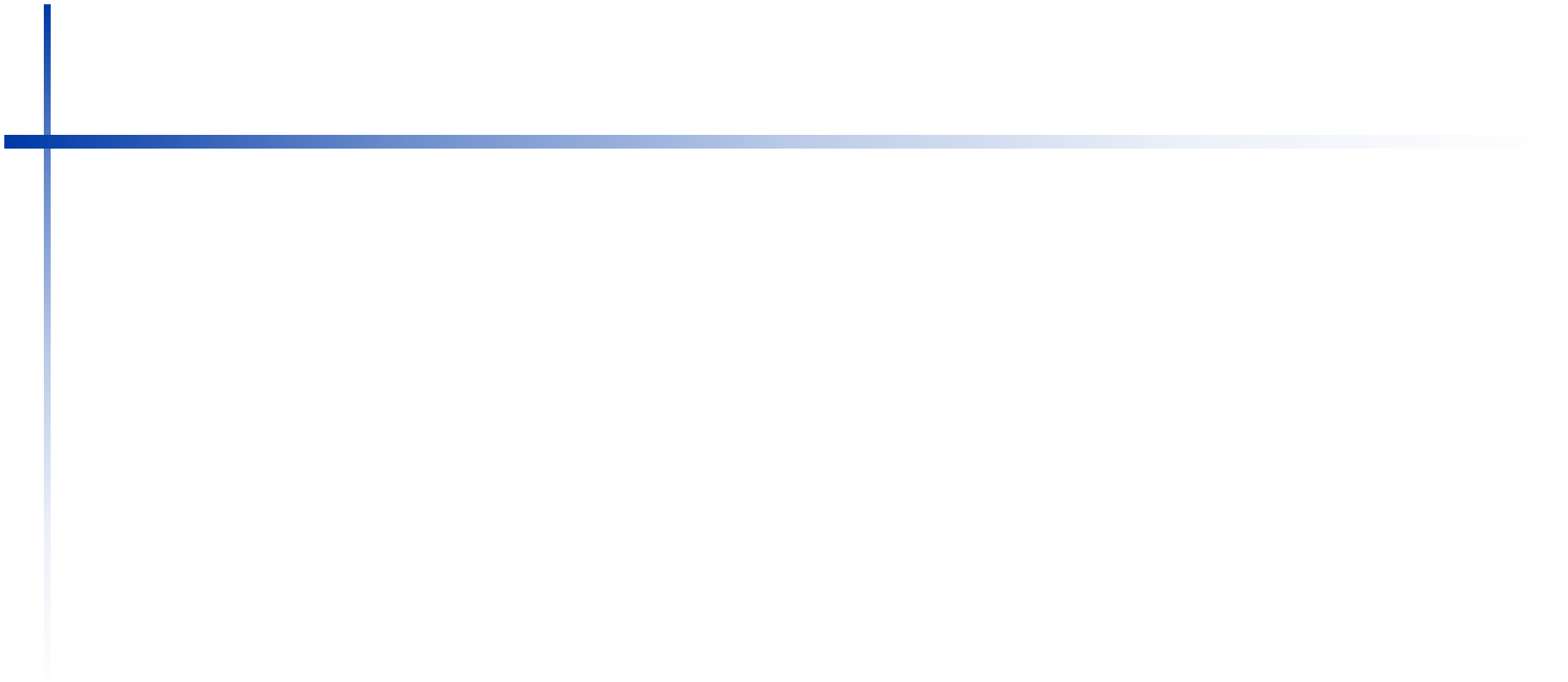
- MIPS: Millions of Instructions Per Second
 - Doesn't account for
 - Differences in ISAs between computers
 - Differences in complexity between instructions

$$\begin{aligned}\text{MIPS} &= \frac{\text{Instruction count}}{\text{Execution time} \times 10^6} \\ &= \frac{\text{Instruction count}}{\frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}} \times 10^6} = \frac{\text{Clock rate}}{\text{CPI} \times 10^6}\end{aligned}$$

- CPI varies between programs on a given CPU

Concluding Remarks

- Cost/performance is improving
 - Due to underlying technology development
- Hierarchical layers of abstraction
 - In both hardware and software
- Instruction set architecture
 - The hardware/software interface
- Execution time: the best performance measure
- Power is a limiting factor
 - Use parallelism to improve performance



SEE YOU NEXT TIME